混合专家模型（MoE）

Mixture of Experts (MoE)

原文作者：@techNmak。

先列专业术语对照表，再提供逐页完整中文译文。每个原文页面的译文之后附原文矢量对照页，完整保留公式、图表、代码和英文引用，便于核对。

正文中文：LXGW WenKai（霞鹜文楷）；英文与数字：Times New Roman。原文对照页保留原始字体。

本文采用翻译辅助服务生成初稿，统一技术术语并检查结构、字体和导航；涉及精确数值、公式或实现细节时，可直接对照相邻原文页。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Mixture of Experts	混合专家	MoE
Routing	路由	
Models	模型	
Matrix Multiplication	矩阵乘法	
Feed-forward Network	前馈网络	FFN
Mixture-of-Experts	混合专家	MoE
Expert Parallelism	专家并行	
Active Parameters	激活参数	
Transformer Block	Transformer 块	
Total Parameters	总参数	
Wall-clock Time	实际耗时	
Expert Capacity	专家容量	
Token Dropping	词元丢弃	
Feed-forward	前馈	
Transformer	Transformer 模型	Transformer
Attention	注意力机制	Attention
Inference	推理	
Training	训练	
Dispatch	分派	
Softmax	Softmax 归一化	Softmax
Experts	专家	
Expert	专家	
Router	路由器	
Token	词元	
FLOPs	浮点运算次数	FLOPs
Model	模型	
Core	核心	
Hidden Representation	隐藏表示	
Sparse Activation	稀疏激活	
Scaling	缩放	
Latency	延迟	
Tokens	词元	

英文原词	中文译名	缩写／符号
Learned Projection	学习得到的投影	
Self-attention	自注意力	
Mental Model	理解框架	
Baseline	基线	
Matrix	矩阵	
SwiGLU	SwiGLU 门控激活	
Representation	表示	
Vector	向量	
Sparse Attention	稀疏注意力	
Dense Attention	稠密注意力	
Normalization	归一化	
Combine	合并	
Values	值	
Logit	逻辑值（归一化前的分数）	
Gradient	梯度	
Value	值	V
Index	索引	
Renormalization	重新归一化	
Serving	推理服务	
Gating	门控	
Load Balancing	负载均衡	
Operator	算子	
Shared Experts	共享专家	
Embeddings	嵌入向量	
Capacity Factor	容量因子	
Residual Path	残差路径	
Residual Connection	残差连接	
Evidence	证据	
Kernels	内核	
Auxiliary Loss	辅助损失	
Load-balancing Loss	负载均衡损失	
Clipping	截断	
Batching	批处理	
Matrices	矩阵	

英文原词	中文译名	缩写／符号
Kernel	内核	
Expert Specialization	专家专门化	
Shared Expert	共享专家	
Memory Footprint	内存占用	
Forward Pass	前向传播	
Precision	精度	
Fused Kernels	融合内核	
Quantization	量化	
Performance	性能	
Arithmetic Intensity	算术强度	
Backward Pass	反向传播	
All-to-all	全对全通信	
Memory Bandwidth	内存带宽	
Residency	驻留状态	
GEMM	通用矩阵乘法	GEMM
Encoder	编码器	
Decoder	解码器	
Mask	掩码	
Batch Size	批大小	
Per-group	逐组	
Resident	驻留	
Activation	激活	
Query	查询	Q
Key	键	K
Memory Traffic	内存流量	

目录导航

点击章节标题跳转到中文译文，点击“原文”核对公式和图表。页码按本译本最终分页生成。

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

混合专家（Mixture of Experts） (教育部) · 第 7 页 ［原文 1］

方向 · 第 9 页 ［原文 2］

1 - 条件计算 · 第 11 页 ［原文 3］

2 - 密集 基线（Baseline） · 第 13 页 ［原文 4］

3 - 萌部层 · 第 15 页 ［原文 5］

4 - Transformer 模型（Transformer） 放置 · 第 17 页 ［原文 6］

5 - 路由器（Router） · 第 19 页 ［原文 7］

6 - TOP-K 路由（Routing） · 第 21 页 ［原文 8］

7 - 已工作 路由（Routing） I · 第 23 页 ［原文 9］

8 - 已使用 路由（Routing） II · 第 25 页 ［原文 10］

9 - 批次 路由（Routing） · 第 27 页 ［原文 11］

10 - 历史 · 第 29 页 ［原文 12］

11 - 喧闹的 TOP-K · 第 31 页 ［原文 13］

12 - GShard 和开关 · 第 33 页 ［原文 14］

13 - TOP-1、TOP-2、TOP-K · 第 35 页 ［原文 15］

14 - 专家容量（Expert Capacity） · 第 37 页 ［原文 16］

15 - 工作能力 · 第 39 页 ［原文 17］

16 - 词元丢弃（Token Dropping） · 第 41 页 ［原文 18］

17 - 路由（Routing） 崩溃 · 第 43 页 ［原文 19］

18 - 早期平衡损失 · 第 45 页 ［原文 20］

19 - 开关平衡损耗 · 第 47 页 ［原文 21］

20 - 路由器（Router） Z 损失 · 第 49 页 ［原文 22］

21 - 其他 路由（Routing） 配方 · 第 51 页 ［原文 23］

22 - 无滴执行 · 第 53 页 ［原文 24］

23 - 细粒度和 共享专家（Shared Experts） · 第 55 页 ［原文 25］

24 - 辅助无损耗平衡 · 第 57 页 ［原文 26］

25 - 总计 VS 激活参数（Active Parameters） · 第 59 页 ［原文 27］

26 - 工作参数示例 · 第 61 页 ［原文 28］

27 - 训练（Training） VS 推理（Inference） · 第 63 页 ［原文 29］

28 - 专家并行（Expert Parallelism） · 第 65 页 ［原文 30］

29 - 性能（Performance） · 第 67 页 ［原文 31］

30 - 专家（Expert） 专业化 · 第 69 页 ［原文 32］

31 - 现代 模型（Models）· 第 71 页 ［原文 33］
32 - 透明实施 · 第 73 页 ［原文 34］
33 - 实施错误 · 第 75 页 ［原文 35］
34 - 常见的误解 · 第 77 页 ［原文 36］
35 - 完整的心理 模型（Model）· 第 79 页 ［原文 37］
参考 · 第 81 页 ［原文 38］

混合专家（Mixture of Experts）(教育部)

对应原文第 1 页；点击查看相邻原文对照页

理解 混合专家（Mixture of Experts） 从稀疏的 路由（Routing） 到现代教育部语言 模型（Models） 面向 AI 工程师、开发人员和认真学习者的简短技术手册 手册 07 - 理解 AI 系列

Understanding Mixture of Experts

From Sparse Routing to Modern MoE Language Models

A short technical handbook for AI engineers, developers, and serious learners

@techNmak

Handbook 07 - Understanding AI Series

方向

对应原文第 2 页；点击查看相邻原文对照页

方向 本手册将阐明什么 混合专家（Mixture-of-Experts） 层很容易总结为“将每个 词元（Token） 路由到几个 专家（Experts）”。这句话是正确的，但它隐藏了几乎所有使现代 MoE 模型（Models） 变得有趣的工程和数学细节。

读完本手册后，您应该能够解释：

- 当Transformer 模型（Transformer） 前馈（Feed-forward） 层变成MoE 层时会发生什么变化？

- 路由器（Router） logits、分数、Top-k 选择和 专家（Expert） 权重如何组合在一起；

- 为什么总参数（Total Parameters）、激活参数（Active Parameters）、浮点运算次数（FLOPs）、重量存储器和实际耗时（Wall-clock Time） 的数量不同；

- 为什么会出现负载不平衡，实际优化了哪些辅助损耗，以及为什么 路由器（Router） 稳定性是一个单独的问题；

- 专家容量（Expert Capacity） 和词元丢弃（Token Dropping） 的含义是什么，以及为什么这两者对于教育部来说都不是基础；

- 为什么分布式MoE需要词元（Token）、分派（Dispatch）、专家（Expert）并行通信以及高效的分组专家（Expert）计算；

- 在不改变核心（Core） MoE 理念的情况下，Mixtral、DeepSeekMoE/DeepSeek-V3 和Qwen3 等设计有何不同。

先决条件 您应该了解 Transformer 块（Transformer Block） 的基本知识，前馈网络（Feed-forward Network）、矩阵乘法（Matrix Multiplication）、Softmax 归一化（Softmax） 的作用以及 训练（Training） 和 推理（Inference） 之间的区别。早期的理解 注意力机制（Attention） 手册是有用的上下文，但 MoE 路由（Routing） 不是 注意力机制（Attention） 机制。

范围 本手册重点介绍现代语言中的稀疏 专家（Expert） 层 - 模型（Model） Transformers。它涵盖了足够的 专家并行（Expert Parallelism） 来解释执行成本，但完整的分布式系统设计属于单独的系统手册。

研究锚点：[1]-[4]、[9]-[16] 理解 混合专家（Mixture of Experts）

ORIENTATION

What this handbook will make clear

A Mixture-of-Experts layer is easy to summarize as “route each token to a few experts.” That sentence is correct, but it hides nearly every engineering and mathematical detail that makes modern MoE models interesting.

By the end of this handbook, you should be able to explain:

- what changes when a Transformer feed-forward layer becomes an MoE layer;
- how router logits, scores, Top-k selection, and expert weighting fit together;
- why total parameters, active parameters, FLOPs, weight memory, and wall-clock time are different quantities;
- why load imbalance appears, what auxiliary losses actually optimize, and why router stability is a separate problem;
- what expert capacity and token dropping mean, and why neither is fundamental to MoE;
- why distributed MoE requires token dispatch, expert-parallel communication, and efficient grouped expert computation;
- how designs such as Mixtral, DeepSeekMoE/DeepSeek-V3, and Qwen3 differ without changing the core MoE idea.

Prerequisites

You should know the basic Transformer block, the role of the feed-forward network, matrix multiplication, softmax, and the difference between training and inference. The earlier *Understanding Attention* handbook is useful context, but MoE routing is not an attention mechanism.

Scope

This handbook focuses on sparse expert layers in modern language-model Transformers. It covers enough expert parallelism to explain execution cost, but full distributed-systems design belongs in a separate systems handbook.

Research anchors: [1]-[4], [9]-[16]

1 - 条件计算

对应原文第 3 页；点击查看相邻原文对照页

1 - 条件计算 为什么存在 混合专家（Mixture of Experts） 密集的神经网络层将相同的学习参数应用于到达它的每个 词元（Token）。因此，增加层宽度或深度会增加参数容量以及对每个 词元（Token） 执行的计算。

混合专家（Mixture of Experts）引入了另一个缩放（Scaling）轴：条件计算。 模型（Model） 可以拥有大量参数，同时仅激活特定输入的子集。中心目标不是“自由参数”。它在某种程度上将 模型（Model） 中存储的参数容量量与应用于一个 词元（Token） 的 专家（Expert） 计算量解耦。

对于 词元（Token） 表示 x，稀疏 专家（Expert） 层从较大的库中选择一小组 专家（Experts）。如果有N个专家（Experts），但只选择了k≪N，则不需要为该词元（Token）执行其他专家（Experts）。

× 路由器（Router） 专家（Expert） 1 专家（Expert） 2

专家（Expert） N 稀疏选择取决于输入。相邻的两个词元（Tokens）可以使用不同的专家（Experts），同一个词元（Token）可以在不同层使用不同的专家（Experts），因为它的隐藏表示（Hidden Representation）通过网络发生变化。

重要： 稀疏激活（Sparse Activation） 减少了执行的 专家（Expert） 计算。它不会自动减少存储所有 专家（Expert） 权重所需的内存，也不会保证较低的实际运行 延迟（Latency）。

研究锚点：[2]、[4]、[7] 理解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

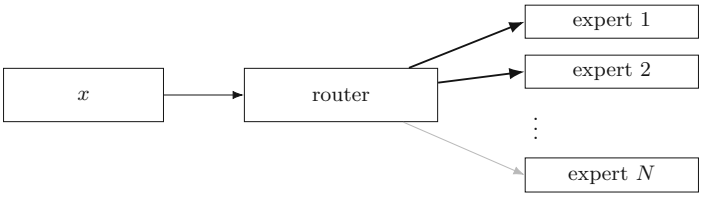
1 - CONDITIONAL COMPUTATION

Why Mixture of Experts exists

A dense neural-network layer applies the same learned parameters to every token that reaches it. Increasing the layer width or depth therefore increases both parameter capacity and the computation performed for every token.

Mixture of Experts introduces another scaling axis: *conditional computation*. The model can own a large bank of parameters while activating only a subset for a particular input. The central goal is not "free parameters." It is to decouple, to some degree, the amount of parameter capacity stored in the model from the amount of expert computation applied to one token.

For a token representation x, a sparse expert layer selects a small set of experts from a larger bank. If there are N experts but only $k \ll N$ are selected, the other experts need not execute for that token.



The sparse choice is input-dependent. Two adjacent tokens can use different experts, and the same token can use different experts at different layers because its hidden representation changes through the network.

Important. Sparse activation reduces which expert computations are executed. It does not automatically reduce the memory required to store all expert weights, nor does it guarantee lower wall-clock latency.

Research anchors: [2], [4], [7]

2 - 密集 基线（Baseline）

对应原文第 4 页；点击查看相邻原文对照页

2 - DENSE 基线（Baseline） 专家（Expert） 所取代的内容 在标准 Transformer 模型（Transformer） 中，前馈（Feed-forward） 子层独立应用于 注意力机制（Attention） 子层之后的每个 词元（Token）。简化的二元 矩阵（Matrix） 前馈网络（Feed-forward Network） 可以写成 FFN(x) = W2phi(W1x)，其中 x ∈ Rdmodel，W1 扩展隐藏维度，phi 是非线性，W2 投影回 模型（Model） 维度。

现代法学硕士经常使用门控变体，例如 SwiGLU 门控激活（SwiGLU），其中涉及额外的 学习得到的投影（Learned Projection）。确切的 FFN 公式是特定于体系结构的。有用的 基线（Baseline） 更简单：密集的 Transformer 模型（Transformer） 在该层具有一组 FFN 参数，并且每个 词元（Token） 使用同一组参数。

MoE Transformer 模型（Transformer） 通常会更改此子层，而不是 注意力机制（Attention） 机制。该层拥有多个具有相同输入/输出形状但参数不同的 FFN，而不是一个 FFN：

路由器（Router） 决定由哪一个处理每个 词元（Token）。

自注意力（Self-attention）

路由器（Router）

理解框架（Mental Model）。在现代稀疏 Transformer 模型（Transformer） MoE 中，“专家（Expert）”通常是层内的 前馈（Feed-forward） 子网络。它不是一个完整的独立语言模型（Model）。

研究锚点：[3]、[4]、[9]、[11] 理解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

2 - THE DENSE BASELINE

What an expert replaces

In a standard Transformer, the feed-forward sublayer is applied independently to every token after the attention sublayer. A simplified two-matrix feed-forward network can be written

$$\mathrm{FFN}(x) = W_2\,\phi(W_1 x),$$

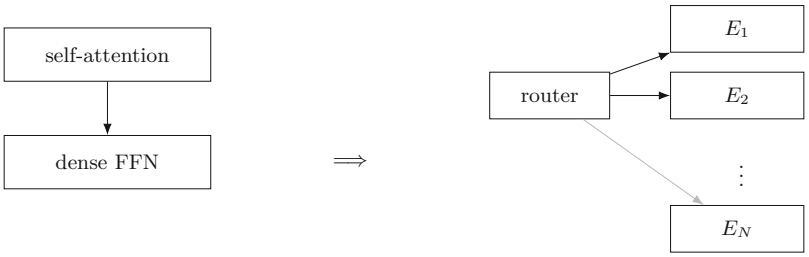
where $x \in \mathbb{R}^{d_{\text{model}}}$, W_1 expands the hidden dimension, ϕ is a nonlinearity, and W_2 projects back to the model dimension.

Modern LLMs often use gated variants such as SwiGLU, which involve an additional learned projection. The exact FFN formula is architecture-specific. The useful baseline is simpler: a dense Transformer has *one set of FFN parameters at that layer*, and every token uses that same set.

An MoE Transformer usually changes this sublayer, not the attention mechanism. Instead of one FFN, the layer owns several FFNs with the same input/output shape but different parameters:

$$E_1(x), E_2(x), \ldots, E_N(x).$$

A router decides which of them process each token.



Mental model. In modern sparse Transformer MoEs, an "expert" is usually a feed-forward subnetwork inside a layer. It is not a complete independent language model.

Research anchors: [3], [4], [9], [11]

3 - 萌部层

对应原文第 5 页；点击查看相邻原文对照页

3 - MOE 层 一个方程，稀疏支持 设 Ei(x) 为 Experti，设 Gi(x) 为 路由器（Router） 的权重。一般的 混合专家（Mixture-of-Experts） 层可以写成 MoE(x) =

方程本身不需要稀疏性。当 G(x) 的大多数条目恰好为零时，计算优势就会显现出来，因此只有一个小的支持集

需要进行评估。

对于 Top-k 词元（Token）-选择 路由（Routing）,|S(x)|正常条件下=k。然后输出是

专家（Experts） 必须产生兼容的输出形状，以便可以组合它们的贡献。原则上，它们可以具有不同的内部架构，但大型语言 模型（Models） 通常使用同质 FFN 专家（Experts），因为正则性简化了 训练（Training） 和执行。

这个表述也解释了教育部没有改变的内容。 词元（Token） 表示（Representation） 在 MoE 边界处仍然具有 模型（Model） 的普通隐藏维度。稀疏性存在于参数选择和 专家（Expert） 执行中，而不是存在于较小的 词元（Token） 向量（Vector） 中。

重要：相同的高级方程式涵盖许多不同的路由器。 Softmax 归一化（Softmax） Top-k、基于 sigmoid 的亲和力、平衡分配和 专家（Expert） 选择方法的不同之处在于 G(x) 的构造方式。该等式是抽象，而不是一种实现的规范。

研究锚点：[2]、[7]、[9]、[10]、[11] 理解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

3 - THE MOE LAYER

One equation, sparse support

Let $E_i(x)$ be expert i and let $G_i(x)$ be the router's weight for that expert. A general mixture-of-experts layer can be written

$$\boxed{\mathrm{MoE}(x) = \sum_{i=1}^{N} G_i(x) E_i(x)}.$$

The equation itself does not require sparsity. The computational advantage appears when most entries of $G(x)$ are exactly zero, so only a small support set

$$\mathcal{S}(x) = \{i : G_i(x) \neq 0\}$$

needs to be evaluated.

For Top-k token-choice routing, $|\mathcal{S}(x)| = k$ under ordinary conditions. The output is then

$$y = \sum_{i \in \mathcal{S}(x)} g_i(x) E_i(x).$$

The experts must produce compatible output shapes so their contributions can be combined. They can, in principle, have different internal architectures, but large language models commonly use homogeneous FFN experts because regularity simplifies training and execution.

This formulation also explains what MoE does *not* change. The token representation still has the model's ordinary hidden dimension at the MoE boundary. Sparsity lives in parameter selection and expert execution, not in a smaller token vector.

Important. The same high-level equation covers many different routers. Softmax Top-k, sigmoid-based affinity, balanced assignment, and expert-choice methods differ in how $G(x)$ is constructed. The equation is the abstraction, not a specification of one implementation.

Research anchors: [2], [7], [9], [10], [11]

4 - Transformer 模型（Transformer） 放置

对应原文第 6 页；点击查看相邻原文对照页

4 - Transformer 模型（Transformer） 放置 MoE 在块中的位置 Transformer 块（Transformer Block） 通常将词元（Token） 混合操作 (注意力机制（Attention）) 与按位置的 前馈（Feed-forward） 操作交替。稀疏 Transformer 模型（Transformer） MoE 模型（Models） 通常用 专家（Expert） 组替换部分或全部密集 FFN 子层。

示意性的预归一化块可以被认为是 h' =h + 注意力机制（Attention）(Norm(h)), h'' =h' + FFN(Norm(h'))。

MoE 版本将第二个计算更改为 h′′ =h′ + MoE(Norm(h′))。

确切的 归一化（Normalization） 和残差排序因 模型（Model） 而异，因此这些方程是结构草图而不是通用实现。

隐藏状态 注意力机制（Attention） 路由器（Router）

加权 合并（Combine） 下一个块 GShard 将稀疏 MoE 层放置在 Transformer 模型（Transformer） 中，并将它们扩展到许多设备上。交换机 Transformer 模型（Transformer） 后来将 词元（Token） 路由（Routing） 简化为 专家（Expert）。 Mixtral 用稀疏的 专家（Expert） 层替换 前馈（Feed-forward） 块，并将每个 词元（Token） 路由到两个 专家（Experts）。

重要： MoE 和 注意力机制（Attention） 稀疏性是不同的想法。 模型（Model） 可以具有 稠密注意力（Dense Attention） 和稀疏 FFN、稀疏注意力（Sparse Attention） 和密集 FFN，或两者兼而有之。

研究锚点：[3]、[4]、[9] 理解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

4 - TRANSFORMER PLACEMENT

Where MoE sits in the block

A Transformer block usually alternates a token-mixing operation (attention) with a position-wise feed-forward operation. Sparse Transformer MoE models normally replace some or all of those dense FFN sublayers with expert banks.

A schematic pre-normalized block can be thought of as

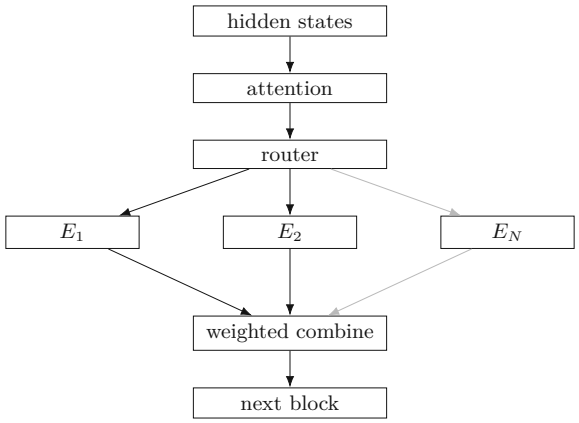
$$h' = h + \text{Attention}(\text{Norm}(h)),$$

$$h'' = h' + \text{FFN}(\text{Norm}(h')).$$

An MoE version changes the second computation to

$$h'' = h' + \text{MoE}(\text{Norm}(h')).$$

The exact normalization and residual ordering varies by model, so these equations are a structural sketch rather than a universal implementation.



GShard placed sparse MoE layers in a Transformer and scaled them across many devices. Switch Transformer later simplified token routing to one expert. Mixtral replaces the feed-forward blocks with sparse expert layers and routes each token to two experts.

Important. MoE and attention sparsity are separate ideas. A model can have dense attention and sparse FFNs, sparse attention and dense FFNs, or both.

Research anchors: [3], [4], [9]

5 - 路由器（Router）

对应原文第 7 页；点击查看相邻原文对照页

5 - 路由器（Router） 从 词元（Token） 向量（Vector） 到 专家（Expert） 分数 对于常见的线性 路由器（Router），采用 词元（Token） 表示 x ∈ Rd 和 路由器（Router） 矩阵（Matrix）

使用 row-向量（Vector） 表示法，路由器（Router） logits 为

每个 专家（Expert） 都会生产一个 逻辑值（归一化前的分数）（Logit）。在 Softmax 归一化（Softmax） 路由器（Router） 中，pi(x) = ehi

j=1ehj

这些 值（Values） 在稀疏化之前形成密集的概率分布。

路由器（Router） 相对于宽 FFN 专家（Expert） 较小，但它不是免费的。它的分数计算随着d×N而增长。

开关 Transformer 模型（Transformer） 显式地注意到此 路由器（Router） 项，同时观察到，当仅选择一个 专家（Expert） 时，专家（Expert） 计算不需要随 专家（Experts） 的总数而增长。

有两个术语细节很重要。

- 路由器（Router） 逻辑值（归一化前的分数）（Logit）：路由（Routing） 网络生成的归一化（Normalization） 之前的分数。

- 门/亲和权重：应用 路由器（Router） 选择的转换后用于对 专家（Experts） 进行排名和/或加权的数量。

这两者并不总是通过 Softmax 归一化（Softmax） 相关。例如，DeepSeek-V3 使用 sigmoid 计算路由 专家（Expert） 亲和力，并对所选组合的亲和力进行标准化。

理解框架（Mental Model）。路由器（Router） 不“理解 专家（Expert） 名称”。它学习从当前 隐藏表示（Hidden Representation） 到数字分数的映射，其 训练（Training） 压力来自任务损失，以及根据设计、平衡/稳定性目标。

研究锚点：[2]、[4]、[11] 理解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

5 - THE ROUTER

From a token vector to expert scores

For the common linear router, take a token representation $x \in \mathbb{R}^d$ and a router matrix

$$W_r \in \mathbb{R}^{d \times N}.$$

Using row-vector notation, the router logits are

$$h = xW_r \in \mathbb{R}^N.$$

One logit is produced for each expert. In a softmax router,

$$p_i(x) = \frac{e^{h_i}}{\sum_{j=1}^{N} e^{h_j}}.$$

These values form a dense probability distribution before sparsification.

The router is small relative to a wide FFN expert, but it is not free. Its score computation grows with $d \times N$. Switch Transformer explicitly notes this router term while observing that the expert computation need not grow with the total number of experts when only one expert is selected.

Two terminology details matter.

- **Router logit**: the pre-normalization score produced by the routing network.
- **Gate/affinity weight**: the quantity used to rank and/or weight experts after applying the router's chosen transformation.

Those two are not always related by softmax. DeepSeek-V3, for example, computes routed-expert affinities with a sigmoid and normalizes selected affinities for combination.

Mental model. The router does not "understand expert names." It learns a mapping from the current hidden representation to numerical scores whose training pressure comes from the task loss and, depending on the design, balancing/stability objectives.

Research anchors: [2], [4], [11]

6 - TOP-K 路由（Routing）

对应原文第 8 页；点击查看相邻原文对照页

6 - TOP-K 路由（Routing） 选择、加权和硬决策 在计算所有 专家（Experts） 的分数后，词元（Token）-选择 路由（Routing） 只保留一小部分。对于 Softmax 归一化（Softmax） Top-k 路由器（Router），定义 S(x) = TopK(p(x),k)。

通用输出是

pi到底是什么？没有单一的普遍答案。

- Shazeer 的稀疏门在未选择的 logits 设置为-∞ 后应用 Softmax 归一化（Softmax）。

- 交换机使用选定的路由器（Router） 概率作为其Top-1 专家（Expert） 的门值（Value）。

- Mixtral 描述了 Top-K logits 上的 Softmax 归一化（Softmax）。

- DeepSeek-V3 使用 sigmoid 相似性，使用 Top-K 进行选择，然后对所选相似性进行标准化。

这就是为什么手册不应该写“Top-k 总是重新规范化原始 Softmax 归一化（Softmax） 概率”作为定义。

如何训练艰难的选择？

索引（Index） 选择本身是离散的。然而，在所选集合不改变的区域内，所选门值（Values）是路由器（Router）分数的可微函数。沙泽尔等人。当k>1时，明确记录所选专家（Experts）的非零导数，并且Switch指出，即使对于Top-1 路由（Routing），其门值（Value）也提供可微分路径。

辅助 路由器（Router） 损耗可以提供额外的 梯度（Gradient） 信号。

在选择边界处，映射是不连续的。在实践中，成功的 MoE 训练（Training） 并不需要假装 Top-k 索引（Index） 操作在任何地方都是平滑的函数。

研究锚点：[2]、[4]、[9]、[11] 理解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

6 - TOP-K ROUTING

Selection, weighting, and the hard decision

After computing scores for all experts, token-choice routing keeps only a small number. For a softmax Top-k router, define

$$\mathcal{S}(x) = \mathrm{TopK}(p(x), k).$$

A generic output is

$$y = \sum_{i \in \mathcal{S}(x)} \tilde{p}_i(x) E_i(x).$$

What exactly is $\tilde{p}_i$? There is no single universal answer.

- Shazeer's sparse gate applies softmax after non-selected logits are set to $-\infty$.
- Switch uses the selected router probability as the gate value for its Top-1 expert.
- Mixtral describes softmax over Top-K logits.
- DeepSeek-V3 uses sigmoid affinities, selects with Top-K, then normalizes the selected affinities.

This is why a handbook should not write "Top-k always renormalizes the original softmax probabilities" as a definition.

How can a hard choice be trained?

The index selection itself is discrete. Within a region where the selected set does not change, however, selected gate values are differentiable functions of the router scores. Shazeer et al. explicitly note nonzero derivatives for selected experts when $k > 1$, and Switch notes that its gate value supplies a differentiable path even with Top-1 routing. Auxiliary router losses can provide additional gradient signals.

At selection boundaries the mapping is discontinuous. In practice, successful MoE training does not require pretending the Top-k index operation is a smooth function everywhere.

Research anchors: [2], [4], [9], [11]

7 - 已工作 路由（Routing） I

对应原文第 9 页；点击查看相邻原文对照页

7 - 已工作 路由（Routing） I 从 logits 到两个选定的 专家（Experts） 考虑四个 专家（Experts） 和一个 词元（Token） 以及 路由器（Router） logits

应用 Softmax 归一化（Softmax） 给出四位小数，

对于 Top-2 路由（Routing），选择 专家（Experts） 1 和 2。

仅对于此有效示例，假设实现重新规范化所选概率，使它们总和为 1：

稀疏门变成

专家（Expert） 逻辑值（归一化前的分数）（Logit） 密 Softmax 归一化（Softmax） 选定重量

该算术显示了两个不同的操作：对 专家（Experts） 进行排名并确定每个选定的 专家（Expert） 贡献多少。

重要：上面的 重新归一化（Renormalization） 步骤是教学性的，不是教育部的通用规则。在为真实架构重现此计算之前，请务必检查 模型（Model） 的 路由器（Router） 定义。

研究锚点：[2]、[9]、[11] 理解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

7 - WORKED ROUTING I

From logits to two selected experts

Consider four experts and one token with router logits

$$h = [2.0,\ 1.2,\ 0.1,\ -0.4].$$

Applying softmax gives, to four decimals,

$$p \approx [0.5919,\ 0.2659,\ 0.0885,\ 0.0537].$$

For Top-2 routing, experts 1 and 2 are selected.

For this worked example only, suppose the implementation renormalizes the selected probabilities so they sum to one:

$$\tilde{p}_1 = \frac{0.5919}{0.5919 + 0.2659} \approx 0.6900,$$

$$\tilde{p}_2 = \frac{0.2659}{0.5919 + 0.2659} \approx 0.3100.$$

The sparse gate becomes

$$G(x) \approx [0.6900,\ 0.3100,\ 0,\ 0].$$

Expert	logit	dense softmax	selected weight
E_1	2.0	0.5919	0.6900
E_2	1.2	0.2659	0.3100
E_3	0.1	0.0885	0
E_4	-0.4	0.0537	0

The arithmetic shows two distinct operations: ranking experts and deciding how much each selected expert contributes.

Important. The renormalization step above is pedagogical, not a universal MoE rule. Always inspect the model's router definition before reproducing this calculation for a real architecture.

Research anchors: [2], [9], [11]

8 - 已使用 路由（Routing） II

对应原文第 10 页；点击查看相邻原文对照页

8 - 已工作 路由（Routing） II 组合 专家（Expert） 输出 使用选定的权重继续前面的示例

假设两个选中的专家（Experts）返回

那么 MoE 的输出是

第一个坐标是

第二个是

这种加权组合就是为什么 Top-2 路由（Routing） 不仅仅是“选择两个 专家（Experts） 并对其进行平均”。 路由器（Router） 可以不均匀地调节它们的贡献。

在 Transformer 模型（Transformer） 中，根据 模型（Model） 的架构，该 专家（Expert） 结果将参与该块的普通残差/归一化（Normalization） 结构。

理解框架（Mental Model）.路由（Routing） 选择计算发生的位置。 专家（Expert） 函数计算候选转换。

门权重决定如何组合选定的转换。

研究锚点：[2]、[9] 理解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

8 - WORKED ROUTING II

Combining the expert outputs

Continue the previous example with selected weights

$$\tilde{p}_1 \approx 0.6900, \qquad \tilde{p}_2 \approx 0.3100.$$

Suppose the two selected experts return

$$E_1(x) = [1.0, -0.5], \qquad E_2(x) = [2.0, 1.5].$$

Then the MoE output is

$$y = 0.6900E_1(x) + 0.3100E_2(x),$$

so

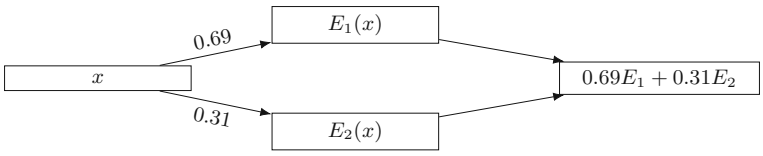
$$y \approx [1.3100,\ 0.1201].$$

The first coordinate is

$$0.6900(1.0) + 0.3100(2.0) = 1.3100,$$

and the second is

$$0.6900(-0.5) + 0.3100(1.5) \approx 0.1201.$$



This weighted combination is why Top-2 routing is not merely "pick two experts and average them." The router can modulate their contributions unequally.

In a Transformer, this expert result then participates in the block's ordinary residual/normalization structure according to that model's architecture.

Mental model. Routing chooses *where* computation happens. Expert functions compute candidate transformations. Gate weights determine how the selected transformations are combined.

Research anchors: [2], [9]

9 - 批次 路由（Routing）

对应原文第 11 页；点击查看相邻原文对照页

9 - BA TCH 路由（Routing） 分配 矩阵（Matrix） 很重要 当我们将许多 词元（Tokens） 布线在一起时，MoE 执行会成为系统问题。对于 T 词元（Tokens） 和 N 专家（Experts），二进制 路由（Routing） 映射可以表示为

其中 Rt,i = 1 表示 tokent 分配给 Experti。

对于四个 词元（Tokens）、四个 专家（Experts） 和 Top-2 路由（Routing），请考虑

每行总和为 2，因为每个 词元（Token） 使用两个 专家（Experts）。列总和给出 专家（Expert） 负载：

总共有 4× 2 = 8 个 专家（Expert） 作业，但作业不平衡。

这个简单的 矩阵（Matrix） 揭示了几个后来的概念：

- 专家（Expert）负载：一列中路由的词元（Tokens）的数量；

- 容量：一个实现允许专家（Expert） 处理多少个任务；

- 分派（Dispatch）：按需要 专家（Experts） 的 词元（Token） 行重新分组；

- 合并（Combine）：将处理后的专家（Expert） 输出返回到原始词元（Token） 订单。

当 专家（Experts） 位于不同设备上时，此逻辑 矩阵（Matrix） 的列对应于可能需要跨网络移动的工作。

重要：如果 模型（Model） 的分配创建了很小的 专家（Expert） 批次、严重不平衡或昂贵的通信，则 模型（Model） 可能在数学上稀疏，同时在操作上效率低下。

研究锚点：[2]、[8]、[16] 理解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

9 - BATCH ROUTING

The assignment matrix matters

MoE execution becomes a systems problem when we route many tokens together. For T tokens and N experts, a binary routing map can be represented as

$$R \in \{0,1\}^{T \times N},$$

where $R_{t,i} = 1$ means token t is assigned to expert i.

For four tokens, four experts, and Top-2 routing, consider

$$R = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}.$$

Each row sums to 2 because each token uses two experts. Column sums give expert loads:

$$[3,\ 2,\ 2,\ 1].$$

There are $4 \times 2 = 8$ expert assignments in total, but the assignments are not balanced.

This simple matrix exposes several later concepts:

- **expert load**: the number of routed tokens in a column;
- **capacity**: how many assignments an implementation allows an expert to process;
- **dispatch**: regrouping token rows by the experts that need them;
- **combine**: returning processed expert outputs to the original token order.

When experts live on different devices, columns of this logical matrix correspond to work that may need to move across the network.

Important. A model may be mathematically sparse while being operationally inefficient if its assignment matrix creates tiny expert batches, severe imbalance, or expensive communication.

Research anchors: [2], [8], [16]

10 - 历史

对应原文第 12 页；点击查看相邻原文对照页

10 - 历史 Y 教育部早于 Transformer 模型（Transformer） 混合专家（Mixture-of-Experts） 的想法比现代法学硕士要古老得多。 Jacobs、Jordan、Nowlan 和 Hinton 在 1991 年引入了局部 专家（Experts） 的自适应混合：单独的 专家（Expert） 网络学习 训练（Training） 案例的不同子集，而 门控（Gating） 机制在其中分配案例。

现代稀疏法学硕士血统增加了两个关键要素。

稀疏条件计算 Shazeer 等人。 (2017) 将 MoE 转变为一个通用神经网络组件，具有潜在的数千个 前馈（Feed-forward） 专家（Experts） 和一个可训练的稀疏门。每个输入仅评估少数 专家（Experts）。他们的语言 模型（Models） 将 MoE 放置在循环层之间，而不是放在 Transformer 模型（Transformer） 内。

然后，Transformer 模型（Transformer） 规模的 MoE GShard 将稀疏门控 MoE 集成到 Transformers 中，并演示了超过 600B 参数的多语言翻译 模型（Model） 的自动分片。交换机Transformer 模型（Transformer）将路由（Routing）简化为单个选定的专家（Expert），并研究了训练（Training）的大规模稳定性、负载平衡和容量。

年份 持久工作理念 1991 Jacobs 等人。在本地学习了 门控（Gating） 专家（Experts） 2017 Shazeer 等人。稀疏 Top-k 条件计算 2020 GShard Transformer 模型（Transformer） MoE 在非常大的分布式规模 2022 Switch Top-1 路由（Routing） 和简化的稀疏执行 2024+ 现代 LLM MoE 新的 专家（Expert） 粒度、平衡和 推理服务（Serving） 设计

理解框架（Mental Model）.MoE 并不是为当今的大型法学硕士发明的技巧。现代 模型（Models） 是从学习模块化到分布式规模稀疏条件计算的漫长演变中的一个阶段。

研究锚点：[1]-[4] 理解 混合专家（Mixture of Experts）

10 - HISTORY

MoE predates the Transformer

Mixture-of-experts ideas are much older than modern LLMs. Jacobs, Jordan, Nowlan, and Hinton introduced *Adaptive Mixtures of Local Experts* in 1991: separate expert networks learned different subsets of training cases while a gating mechanism allocated cases among them.

The modern sparse-LLM lineage adds two crucial ingredients.

Sparse conditional computation

Shazeer et al. (2017) turned MoE into a general-purpose neural-network component with potentially thousands of feed-forward experts and a trainable sparse gate. Only a handful of experts were evaluated for each input. Their language models placed the MoE between recurrent layers rather than inside a Transformer.

Transformer-scale MoE

GShard then integrated sparsely gated MoE into Transformers and demonstrated automatic sharding of a multilingual translation model beyond 600B parameters. Switch Transformer simplified routing to a single selected expert and studied training stability, load balance, and capacity at large scale.

Year	Work	Durable idea
1991	Jacobs et al.	learned gating across local experts
2017	Shazeer et al.	sparse Top-k conditional computation
2020	GShard	Transformer MoE at very large distributed scale
2022	Switch	Top-1 routing and simplified sparse execution
2024+	modern LLM MoEs	new expert granularity, balancing, and serving designs

Mental model. MoE is not a trick invented for today's giant LLMs. Modern models are one stage in a longer evolution from learned modularity to sparse conditional computation at distributed scale.

Research anchors: [1]-[4]

11 - 喧闹的 TOP-K

对应原文第 13 页；点击查看相邻原文对照页

11 - NOISY TOP-K 2017 年稀疏门控公式 Shazeer 等人。提出了一个嘈杂的 Top-k 门。在他们的符号中，密集预门 值（Value） 是 Hi(x) = (xWg)i +ϵi Softplus((xWnoise)i)，具有高斯噪声 ϵi。 Top-k 算子（Operator） 保留最大的 k 值（Values） 并将其余设置为-∞，然后 Softmax 归一化（Softmax） 生成稀疏门：

G(x) = Softmax 归一化（Softmax）(KeepTopK(H(x),k))。

因此，未选择的 专家（Experts） 接收的门权重为零，并且不需要针对该输入执行。

噪声在原始设计中服务于 训练（Training） 目的，包括支持 负载均衡（Load Balancing） 和探索。不应将其概括为“所有 MoE 路由器都会增加噪音”。许多后来的路由器使用不同的机制。

同一篇论文强调了仍然相关的两个系统成本：

- 缩小专家（Expert） 批次：批次大小b、N 专家（Experts） 和每个词元（Token） 的k 分配，平均专家（Expert） 在分布式聚合之前看到大约kb/N 词元（Token） 分配；

- 网络带宽：如果专家（Experts） 是分布式的，则词元（Token） 表示和专家（Expert） 输出必须移至/移出持有这些专家（Experts） 的设备。

这些不是偶然的实施错误。它们直接来自稀疏的、依赖于输入的 路由（Routing）。

重要：最初的 2017 年门是基础性的，但现代路由器应该根据它们自己的定义来描述，而不是假设为嘈杂的 Top-k 副本。

研究锚：[2]

了解 混合专家（Mixture of Experts）

11 - NOISY TOP-K

The 2017 sparsely-gated formulation

Shazeer et al. proposed a noisy Top-k gate. In their notation, a dense pre-gate value is

$$H_i(x) = (xW_g)_i + \epsilon_i \text{ Softplus}((xW_{noise})_i),$$

with Gaussian noise ϵ_i. A Top-k operator keeps the largest k values and sets the rest to $-\infty$, then softmax produces the sparse gate:

$$G(x) = \text{Softmax}(\text{KeepTopK}(H(x), k)).$$

Thus non-selected experts receive exactly zero gate weight and do not need to execute for that input.

The noise served a training purpose in the original design, including support for load balancing and exploration. It should not be generalized into "all MoE routers add noise." Many later routers use different mechanisms.

The same paper highlighted two systems costs that are still relevant:

- **shrinking expert batches**: with batch size b, N experts, and k assignments per token, the average expert sees roughly kb/N token assignments before distributed aggregation;
- **network bandwidth**: if experts are distributed, token representations and expert outputs have to move to/from the devices holding those experts.

Those are not incidental implementation bugs. They follow directly from sparse, input-dependent routing.

Important. The original 2017 gate is foundational, but modern routers should be described from their own definitions rather than assumed to be noisy Top-k copies.

Research anchors: [2]

12 - GShard 和开关

对应原文第 14 页；点击查看相邻原文对照页

12 - GSHARD 和 SWITCH 稀疏 专家（Experts） 如何进入大型 Transformer GShard 在 Transformer 模型（Transformer） 内使用稀疏 MoE 并将其与自动分片相结合。其报告的系统在 2048 个 TPU v3 加速器上将多语言翻译 模型（Model） 扩展到超过 600B 参数。持久的架构点比标题数字更重要：专家（Expert） 参数可以跨设备分布，同时仅为每个 词元（Token） 执行选定的 专家（Expert） 路径。

交换机 Transformer 模型（Transformer） 进一步简化了 路由（Routing） 算法。每个 词元（Token） 都被发送到具有最高 路由器（Router） 概率的单个 专家（Expert），而不是 Top-2。作者报告了三个实际动机：减少 路由（Routing） 工作、减少 专家容量（Expert Capacity） 要求以及降低通信复杂性。

对于 Top-1 开关 路由（Routing），y =pi* (x)Ei* (x)，i* = arg max

所选概率仍然乘以 专家（Expert） 输出，因此 路由器（Router） 具有可微分门 - 值（Value） 路径，尽管 i* 的身份是一个艰难的选择。

Switch 还使 专家容量（Expert Capacity） 和 词元（Token） 溢出了核心工程概念，因为其面向 TPU 的实现使用了静态边界的 专家（Expert） 批次。

重要：“Top-1 更简单”与“Top-1 普遍更好”并不是同一说法。后来成功的 模型（Models） 使用 Top-2、Top-8 和其他 路由（Routing） 预算。k 是一种架构和系统选择。

研究锚点：[3]、[4] 理解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

12 - GSHARD AND SWITCH

How sparse experts entered large Transformers

GShard used sparse MoE inside a Transformer and combined it with automatic sharding. Its reported system scaled a multilingual translation model beyond 600B parameters across 2048 TPU v3 accelerators. The lasting architectural point is more important than that headline number: expert parameters could be distributed across devices while only selected expert paths executed for each token.

Switch Transformer simplified the routing algorithm further. Instead of Top-2, each token was sent to the single expert with the highest router probability. The authors reported three practical motivations: less routing work, smaller expert capacity requirements, and reduced communication complexity.

For Top-1 Switch routing,

$$y = p_{i^*}(x)E_{i^*}(x), \qquad i^* = \arg\max_i p_i(x).$$

The selected probability still multiplies the expert output, so the router has a differentiable gate-value path even though the identity of i^* is a hard choice.

Switch also made expert capacity and token overflow central engineering concepts because its TPU-oriented implementation used statically bounded expert batches.

Important. "Top-1 is simpler" is not the same claim as "Top-1 is universally better." Later successful models use Top-2, Top-8, and other routing budgets. k is an architectural and systems choice.

Research anchors: [3], [4]

13 - TOP-1、TOP-2、TOP-K

对应原文第 15 页；点击查看相邻原文对照页

13 - TOP-1、TOP-2、TOP-K 路由（Routing） 预算更改多个数字 所选专家的数量以多种方式影响 专家（Expert） 路径。

路由（Routing） 每 词元（Token） 专家（Expert） 工作 典型结果 Top-1 一条 专家（Expert） 路径更简单 分派（Dispatch），最小 专家（Expert） 工作预算 Top-2 两条 专家（Expert） 路径 两种转换的混合，更多分配 Top-k k 专家（Expert） 路径更高专家（Expert） 工作/通信要求增长 如果 专家（Expert） 宽度保持固定，从 Top-1 移动到 Top-2 大约会使该 MoE 层的 专家（Expert）-FFN 计算量增加一倍。它不一定是整个 模型（Model） 浮点运算次数（FLOPs） 的两倍，因为 注意力机制（Attention）、嵌入向量（Embeddings）、归一化（Normalization）、共享专家（Shared Experts） 和其他层仍然存在。

同样，如果 专家（Expert） 总数 N 增长，而 k 和 专家（Expert） 形状保持固定，则存储的 专家（Expert） 参数总数会大幅增长，而不会按比例增加一个 词元（Token） 的 专家（Expert） 工作。但是 路由器（Router） 得分随着 N 的增长而增长，仍然需要存储/分片更大的权重库，并且分布式执行会变得更加困难。

现代示例使多样性具体化：Switch 使用 Top-1，Mixtral 8x7B 使用 Top-2，Qwen3-30B-A3B 报告 128 专家（Experts），其中 8 个已激活，而 DeepSeek-V3 选择 8 个路由 专家（Experts） 及其共享 专家（Expert） 路径。

理解框架（Mental Model）.N 控制 专家（Expert） 组的大小。k 控制 词元（Token） 使用该组的多少。这两个数字都无法单独告诉您 延迟（Latency）、内存或 模型（Model） 质量。

研究锚点：[4]、[9]、[11]、[14] 理解 混合专家（Mixture of Experts）

13 - TOP-1, TOP-2, TOP-K

The routing budget changes more than one number

The number of selected experts k affects the expert path in several ways.

Routing	Per-token expert work	Typical consequence
Top-1	one expert path	simpler dispatch, smallest expert-work budget
Top-2	two expert paths	mixture of two transformations, more assignments
Top-k	k expert paths	higher expert work/communication as k grows

If expert widths are held fixed, moving from Top-1 to Top-2 approximately doubles the expert-FFN computation for that MoE layer. It does *not* necessarily double whole-model FLOPs because attention, embeddings, normalization, shared experts, and other layers remain.

Likewise, if total expert count N grows while k and expert shape stay fixed, total stored expert parameters can grow substantially without proportionally increasing the expert work for one token. But router scoring grows with N, the larger weight bank still has to be stored/sharded, and distributed execution can become harder.

Modern examples make the diversity concrete: Switch uses Top-1, Mixtral 8x7B uses Top-2, Qwen3-30B-A3B reports 128 experts with 8 activated, and DeepSeek-V3 selects 8 routed experts plus its shared-expert path.

Mental model. N controls the size of the expert bank. k controls how much of that bank a token uses. Neither number alone tells you latency, memory, or model quality.

Research anchors: [4], [9], [11], [14]

14 - 专家容量（Expert Capacity）

对应原文第 16 页；点击查看相邻原文对照页

14 - EXPER T CAPACITY 静态执行约束，不是 MoE 的定义 Dynamic 路由（Routing） 可以将奇数个 词元（Tokens） 发送到不同的 专家（Experts）。因此，一些实现为每个 专家（Expert） 提供固定的处理能力。

对于 Top-1 开关 Transformer 模型（Transformer） 实现，专家容量（Expert Capacity） 定义为

其中，T 是 路由（Routing） 组中 词元（Tokens） 的编号，N 是 专家（Experts） 的编号，cf 是 容量因子（Capacity Factor）。

Atcf = 1，专家（Expert） 插槽总数恰好等于此 Top-1 设置中的 词元（Token） 计数。高于一的因素会产生缓冲

这个公式经常被过于广泛地重复。对于一般的 Top-k 路由（Routing），有 kT 词元（Token）-专家（Expert） 分配，并且框架/论文在如何定义容量、分组、填充以及是否将 k 折叠到容量公式中方面有所不同。一些现代实现完全避免固定 per-专家容量（Expert Capacity）。

重要： 专家容量（Expert Capacity） 是与特定 路由（Routing）/运行时设计相关的执行策略。 MoE 在数学上需要稀疏的 专家（Expert） 选择，而不是固定容量的缓冲区。

能力也是有范围的。它可以按批次、每个本地 路由（Routing） 组、每个 专家（Expert） 并行等级或其他单位进行计算。该范围改变了分配器可见的不平衡情况。

研究锚点：[4]、[7]、[16] 理解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

14 - EXPERT CAPACITY

A static execution constraint, not the definition of MoE

Dynamic routing can send uneven numbers of tokens to different experts. Some implementations therefore give each expert a fixed processing capacity.

For the Top-1 Switch Transformer implementation, expert capacity is defined as

$$\boxed{C = \frac{T}{N} \times c_f}$$

where T is the number of tokens in the routing group, N is the number of experts, and c_f is the capacity factor.

At $c_f = 1$, total expert slots exactly equal the token count in this Top-1 setting. A factor above one creates buffer for uneven routing, but can introduce padding, extra memory, and extra work in statically shaped implementations.

This formula is often repeated too broadly. For general Top-k routing there are kT token-expert assignments, and frameworks/papers differ in how they define capacity, grouping, padding, and whether k is folded into the capacity formula. Some modern implementations avoid fixed per-expert capacity entirely.

Important. Expert capacity is an execution policy associated with particular routing/runtime designs. MoE mathematically requires sparse expert selection, not a fixed-capacity buffer.

Capacity also has a scope. It may be computed per batch, per local routing group, per expert-parallel rank, or another unit. That scope changes which imbalances are visible to the allocator.

Research anchors: [4], [7], [16]

15 - 工作能力

对应原文第 17 页；点击查看相邻原文对照页

15 - 工作容量 容量因子（Capacity Factor） 实际购买什么 采取 Top-1 Switch 式 路由（Routing） 组

均匀平衡的负载将为 16 = 64 词元（Tokens）/专家（Expert）。

配置容量为

在所有 专家（Experts） 中，运行时都有空间

专家（Expert） 插槽，即使仅存在 1024 个 词元（Token） 分配。 25% 的额外时段预算是防止失衡的缓冲。

假设 专家（Expert） 3 收到 95 词元（Tokens）。在固定容量的 Switch 行为下，只能容纳 80 个。剩余的

词元（Tokens） 溢出 专家（Expert） 的容量。在 Switch 中，溢出的 词元（Tokens） 会跳过 专家（Expert） 计算并继续执行 残差路径（Residual Path）。

增加 cf 会减少溢出的可能性，但会增加静态配置的 专家（Expert） 工作/内存量。减少它可以节省容量，但更容易溢出。

理解框架（Mental Model）.容量因子（Capacity Factor） 是备用 专家（Expert） 插槽与不平衡后果之间的系统权衡。

它不是具有通用最佳 值（Value） 的优质旋钮。

重要：这个工作示例是 Top-1 和 Switch 特定的。不要在每个 Top-k MoE 实现中盲目地重复使用 T/N×cf。

研究锚：[4]

了解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

15 - WORKED CAPACITY

What a capacity factor actually buys

Take a Top-1 Switch-style routing group with

$$T = 1024 \text{ tokens}, \qquad N = 16 \text{ experts}, \qquad c_f = 1.25.$$

The evenly balanced load would be

$$\frac{1024}{16} = 64 \text{ tokens/expert}.$$

The configured capacity is

$$C = 64 \times 1.25 = 80 \text{ tokens/expert}.$$

Across all experts, the runtime has room for

$$16 \times 80 = 1280$$

expert slots, even though only 1024 token assignments exist. That 25% extra slot budget is the buffer against imbalance.

Suppose expert 3 receives 95 tokens. Under the fixed-capacity Switch behavior, only 80 fit. The remaining

$$95 - 80 = 15$$

tokens overflow that expert's capacity. In Switch, overflowed tokens skip the expert computation and continue through the residual path.

Increasing c_f would reduce the chance of overflow, but increases the amount of statically provisioned expert work/memory. Decreasing it saves capacity but makes overflow easier.

Mental model. Capacity factor is a systems trade-off between spare expert slots and the consequences of imbalance. It is not a quality knob with a universal optimal value.

Important. This worked example is Top-1 and Switch-specific. Do not reuse $T/N \times c_f$ blindly for every Top-k MoE implementation.

Research anchors: [4]

16 - 词元丢弃（Token Dropping）

对应原文第 18 页；点击查看相邻原文对照页

16 - 词元丢弃（Token Dropping） 溢出行为是一种设计选择 较旧的固定容量 MoE 实现使 词元丢弃（Token Dropping） 可见，因为静态 专家（Expert） 缓冲区可能会溢出。

在交换机 Transformer 模型（Transformer） 中，溢出的 词元（Token） 不会接收稀疏 专家（Expert） 计算，而是通过 残差连接（Residual Connection） 向前传递。

这段历史有时会变成虚假陈述“教育部放弃了 词元（Tokens）”。现代 证据（Evidence） 说明了为什么这是错误的。

MegaBlocks MegaBlocks 将 专家（Expert） 计算重新表述为块稀疏操作，可以有效地处理不平等的 专家（Expert） 负载。其设计明确避免了在删除 词元（Tokens） 和将所有 专家（Experts） 填充到固定容量之间进行选择。

DeepSeek-V3 DeepSeek-V3 报告指出，其 负载均衡（Load Balancing） 足够有效，不会在 训练（Training） 期间丢弃 词元（Tokens），并描述了旨在避免在 推理（Inference） 期间丢弃 词元（Tokens） 的部署策略。

当前 Megatron 核心（Core） 当前 NVIDIA Megatron 核心（Core） 文档将 moe_专家（Expert）_capacity_factor=None 定义为默认情况下没有 词元丢弃（Token Dropping）。当静态形状或其他执行要求使容量限制模式有用时，容量限制模式仍然可用。

重要：“无滴”并不意味着“无成本”。变量 专家（Expert） 负载仍然必须有效地表示、通信、调度和计算。

持久的区别是：

路由（Routing）不平衡≠强制词元（Token）损失。

运行时可以通过动态缓冲区、稀疏 内核（Kernels）、填充、容量限制或其他策略来处理不平衡。

研究锚点：[4]、[8]、[11]、[16] 理解 混合专家（Mixture of Experts）

16 - TOKEN DROPPING

Overflow behavior is a design choice

Older fixed-capacity MoE implementations made token dropping visible because static expert buffers could overflow. In Switch Transformer, an overflowed token does not receive that sparse expert computation and is passed onward through the residual connection.

That history sometimes becomes the false statement "MoE drops tokens." Modern evidence shows why this is wrong.

MegaBlocks

MegaBlocks reformulated expert computation as block-sparse operations that can efficiently handle unequal expert loads. Its design explicitly avoids the choice between dropping tokens and padding all experts to a fixed capacity.

DeepSeek-V3

The DeepSeek-V3 report states that its load balancing is effective enough that it does not drop tokens during training, and describes deployment strategies intended to avoid dropping during inference as well.

Current Megatron Core

Current NVIDIA Megatron Core documentation defines `moe_expert_capacity_factor=None` as no token dropping by default. Capacity-limited modes remain available when static shapes or other execution requirements make them useful.

Important. "Dropless" does not mean "costless." Variable expert loads still have to be represented, communicated, scheduled, and computed efficiently.

The durable distinction is:

$$\text{routing imbalance} \neq \text{mandatory token loss.}$$

A runtime can handle imbalance through dynamic buffers, sparse kernels, padding, capacity limits, or other strategies.

Research anchors: [4], [8], [11], [16]

17 - 路由（Routing） 崩溃

对应原文第 19 页；点击查看相邻原文对照页

17 - 路由（Routing） 崩溃 为什么负载不平衡会自我强化 可训练的 路由器（Router） 可以自由地选择某些 专家（Experts） 而不是其他 专家（Experts）。如果没有反压力，这种偏好就会变得极端。

沙泽尔等人。描述了一种自我强化的故障模式：少数受青睐的 专家（Experts） 收到更多示例，因此收到更多任务 梯度（Gradient） 更新，改进得更快，并对 路由器（Router） 变得更具吸引力。其他 专家（Experts） 可能训练不足。

负载不平衡会产生两个不同的问题。

优化问题 如果大多数数据流经 专家（Experts） 的一小部分，则 模型（Model） 无法有效地使用其拥有的参数容量。未充分使用的 专家（Experts） 从主要任务中接收到的学习信号较少。

系统问题 当 专家（Experts） 位于不同设备上时，步长可能会受到负载最重的 专家（Expert）/rank 的限制。一些设备在其他设备处理大批量 词元（Token） 时等待。容量有限的实现可能会溢出或填充。

动态实现仍然面临落后者和不规则批量大小的问题。

这两个问题解释了为什么 负载均衡（Load Balancing） 不仅仅是对相等直方图的装饰性偏好。

理解框架（Mental Model）.Sparse 路由（Routing） 在 专家（Expert） 学习和 专家（Expert） 被选择之间创建一个反馈循环。

平衡方法试图防止该循环崩溃到太少的 专家（Experts） 上，而不压倒主要语言 - 模型（Model） 目标。

重要：完全统一的 路由（Routing） 本身并不是学习目标。有用时，仍应允许 模型（Model） 以不同方式使用 专家（Experts）。挑战在于防止病理性失衡，同时保持有用的专业化。

研究锚点：[2]、[4]、[11] 理解 混合专家（Mixture of Experts）

17 - ROUTING COLLAPSE

Why load imbalance can reinforce itself

A trainable router is free to prefer some experts over others. Without counter-pressure, that preference can become extreme.

Shazeer et al. described a self-reinforcing failure mode: a few favored experts receive more examples, therefore receive more task-gradient updates, improve faster, and become even more attractive to the router. Other experts may be under-trained.

Load imbalance creates two different problems.

Optimization problem

If most data flows through a small subset of experts, the model is not effectively using the parameter capacity it owns. Under-used experts receive less learning signal from the main task.

Systems problem

When experts are on different devices, the step can be limited by the most heavily loaded expert/rank. Some devices wait while others process large token batches. Capacity-limited implementations may overflow or pad. Dynamic implementations still suffer stragglers and irregular batch sizes.

These two problems explain why load balancing is not merely a cosmetic preference for equal histograms.

Mental model. Sparse routing creates a feedback loop between *which expert learns* and *which expert is chosen*. Balancing methods try to keep that loop from collapsing onto too few experts without overwhelming the main language-model objective.

Important. Perfectly uniform routing is not itself the learning objective. The model should still be allowed to use experts differently when useful. The challenge is preventing pathological imbalance while preserving useful specialization.

Research anchors: [2], [4], [11]

18 - 早期平衡损失

对应原文第 20 页；点击查看相邻原文对照页

18 - 早期平衡损失 重要性和负荷不是同一个量 2017 年教育部的论文门控稀疏，区分了两个容易混淆的概念。

对于batchX，它将expertimportance定义为门值（Values）的总和：

重要性(X) =

辅助损失（Auxiliary Loss） 惩罚了那些 专家（Expert） 重要性 值（Values） 的变异系数的平方：

L重要性=w重要性CV(重要性(X))2。

这里 CV = 标准差/平均值。

为什么还需要另一次损失？门总重量相等并不能保证 词元（Token） 计数相等。一台 专家（Expert） 可以收到一些大重量的 词元（Tokens），而另一台则收到许多小重量的 词元（Tokens）。即使总门质量看起来平衡，硬件也会看到不同数量的 词元（Token） 行。

沙泽尔等人。因此，还引入了一个面向负载的目标，与噪声门下的预期 专家（Expert） 分配相关。它的确切结构取决于嘈杂的 Top-k 公式。

重要：这是一个有用的历史教训：“平衡 路由器（Router）”未指定。您必须询问方法是否平衡概率质量、硬分配计数、设备负载、序列局部负载或其他。

后来的工作简化或取代了这些目标。 Switch 使用涉及硬分配分数和平均 路由器（Router） 概率的单个可微代理。 DeepSeek-V3 使用动态 路由（Routing） 偏差作为其主要批次级平衡压力。

研究锚点：[2]、[4]、[11] 理解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

18 - EARLY BALANCING LOSSES

Importance and load are not the same quantity

The 2017 sparsely gated MoE paper separated two notions that are easy to conflate.

For a batch X, it defined expert *importance* as the sum of gate values:

$$\text{Importance}(X) = \sum_{x \in X} G(x).$$

An auxiliary loss penalized the squared coefficient of variation of those expert importance values:

$$L_{\text{importance}} = w_{\text{importance}} \, \text{CV}(\text{Importance}(X))^2.$$

Here CV = standard deviation/mean.

Why was another loss needed? Equal total gate weight does not guarantee equal token counts. One expert could receive a few tokens with large weights while another receives many tokens with small weights. The hardware sees different numbers of token rows even if summed gate mass looks balanced.

Shazeer et al. therefore also introduced a load-oriented objective tied to expected expert assignment under their noisy gate. Its exact construction depends on that noisy Top-k formulation.

Important. This is a useful historical lesson: "balance the router" is underspecified. You must ask whether a method balances probability mass, hard assignment counts, device load, sequence-local load, or something else.

Later work simplified or replaced these objectives. Switch uses a single differentiable proxy involving hard assignment fractions and average router probabilities. DeepSeek-V3 uses dynamic routing biases for its main batch-level balancing pressure.

Research anchors: [2], [4], [11]

19 - 开关平衡损耗

对应原文第 21 页；点击查看相邻原文对照页

19 - 开关平衡损耗 朝向均匀 专家（Expert） 使用的可微分压力 对于 Top-1 路由（Routing），开关 Transformer 模型（Transformer） 定义 Laux =αN

菲皮，其中

是实际发送给 Experti 的 词元（Tokens） 的分数，并且

是分配给该 专家（Expert） 的平均 路由器（Router） 概率。

硬分数 fi 是不可微分的，但 Pi 是可微分的。该产品将观察到的 路由（Routing） 负载与可微概率信号耦合。

考虑四个 专家（Experts）

然后

fiPi = 4(0.20 + 0.075 + 0.05) = 1.30。

fiPi = 1.00。

因此，不平衡示例中的加权损失为 1.30α，而均匀示例中的加权损失为 α。

重要：统一引用 值（Value） 不为零。当 专家（Expert） 计数发生变化时，factorN 保持统一情况的比例不变。

这个损失是为 Switch 的 Top-1 设置而设计的。在未检查该架构如何定义分配和平衡的情况下，请勿将该方程移植到另一个 路由（Routing） 方案中。

研究锚：[4]

了解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

19 - SWITCH BALANCING LOSS

A differentiable pressure toward uniform expert use

For Top-1 routing, Switch Transformer defines

$$L_{\text{aux}} = \alpha N \sum_{i=1}^{N} f_i P_i,$$

where

$$f_i = \frac{1}{T} \sum_{x \in B} \mathbf{1}\{\arg\max p(x) = i\}$$

is the fraction of tokens actually dispatched to expert i, and

$$P_i = \frac{1}{T} \sum_{x \in B} p_i(x)$$

is the average router probability assigned to that expert.

The hard fraction f_i is not differentiable, but P_i is. The product couples the observed routing load to a differentiable probability signal.

Consider four experts with

$$f = [0.50, 0.25, 0.25, 0], \qquad P = [0.40, 0.30, 0.20, 0.10].$$

Then

$$N \sum_i f_i P_i = 4(0.20 + 0.075 + 0.05) = 1.30.$$

For perfectly uniform $f = P = [0.25, 0.25, 0.25, 0.25]$,

$$N \sum_i f_i P_i = 1.00.$$

So the weighted loss is 1.30α in the imbalanced example versus α in the uniform example.

Important. The uniform reference value is not zero. The factor N keeps the scale of the uniform case constant as expert count changes.

This loss was designed for Switch's Top-1 setting. Do not transplant the equation into another routing scheme without checking how that architecture defines assignments and balance.

Research anchors: [4]

20 - 路由器（Router） Z 损失

对应原文第 22 页；点击查看相邻原文对照页

20 - 路由器（Router） Z-LOSS 稳定性与负载平衡是不同的问题 MoE 路由器（Router） 包含指数和硬 路由（Routing） 决策，因此大的 路由器（Router） logits 可能会产生数值和 训练（Training） 不稳定问题。 ST-MoE 推出 路由器（Router） z 损耗

日志

这种损失会惩罚大型对数归一化器，并鼓励 路由器（Router） 对数保持较小的幅度。 ST-MoE 报告称，这提高了 训练（Training） 在其实验中的稳定性，而没有出现更严格的更新 截断（Clipping） 中出现的质量损失。

该工作中的总 训练（Training） 目标包括主要交叉熵损失、负载平衡项以及具有单独系数的 路由器（Router） z 损失：

L总=LCE+cBLB+czLz。

这种区别很重要：

- 负载平衡损耗决定了词元（Tokens） 在专家（Experts） 上的分布方式；

- 路由器（Router） z 损失约束路由器（Router）-逻辑值（归一化前的分数） （Logit） 规模的稳定性。

它们可以共存，因为它们解决不同的问题。

重要：将 z-loss 称为“另一个 负载均衡损失（Load-balancing Loss）”就抹杀了它的目的。 路由器（Router） 可能在数值上表现良好但仍然不平衡，或者平衡但仍遭受不稳定的 逻辑值（归一化前的分数） （Logit） 幅度。

研究锚：[7]

了解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

20 - ROUTER Z-LOSS

Stability is a different problem from load balance

An MoE router contains exponentials and hard routing decisions, so large router logits can create numerical and training-instability problems. ST-MoE introduced the router z-loss

$$L_z(x) = \frac{1}{B}\sum_{i=1}^{B}\left(\log\sum_{j=1}^{N} e^{x_j^{(i)}}\right)^2$$

for a batch of B token router-logit vectors over N experts.

The loss penalizes large log-normalizers and encourages router logits to remain smaller in magnitude. ST-MoE reports that this improved training stability in its experiments without the quality loss seen with tighter update clipping.

The total training objective in that work includes the main cross-entropy loss, a load-balance term, and the router z-loss with separate coefficients:

$$L_{\text{total}} = L_{\text{CE}} + c_B L_B + c_z L_z.$$

This distinction matters:

- a **load-balancing loss** shapes how tokens are distributed across experts;
- the **router z-loss** constrains router-logit scale for stability.

They can coexist because they solve different problems.

Important. Calling z-loss "another load-balancing loss" erases its purpose. A router can be numerically well-behaved and still imbalanced, or balanced while still suffering unstable logit magnitudes.

Research anchors: [7]

21 - 其他 路由（Routing） 配方

对应原文第 23 页；点击查看相邻原文对照页

21 - 其他 路由（Routing） 公式 词元（Token） 选择是常见的，非强制性 Top-k 词元（Token）-选择 路由（Routing） 询问每个 词元（Token） 它想要哪个 专家（Experts）。其他工作改变了分配问题本身。

BASE 层 BASE 将 词元（Token） 到 专家（Expert） 分配制定为平衡线性分配问题。其设计强制每个 专家（Expert） 的 词元（Tokens） 数量相等，因此避免了该配方中的辅助平衡损失。

专家（Expert） 选择 专家（Expert） 选择反转方向：每个 专家（Expert） 选择其得分最高的 词元（Tokens） 直至固定的桶大小。这通过构造固定了每个 专家容量（Expert Capacity），而 词元（Token） 可以通过可变数量的 专家（Experts） 来选择。

方法 谁选择？负载属性 词元（Token）-选择 Top-k 词元（Token） 选择sk 专家（Experts） 专家（Expert） 负载可以变化 BASE 全局平衡分配 通过结构均衡分配 专家（Expert） 选择 专家（Expert） 选择顶部 词元（Tokens） 固定 专家（Expert） 桶，变量专家（Experts）/- 词元（Token） 这些方法说明了一个更广泛的观点：稀疏 MoE 是一系列条件计算架构，而不是一个不可变的 路由器（Router） 算法。

重要提示：路由（Routing） 表述会改变语义、优化和执行。完美平衡负载的替代方案可能会对 词元（Token） 覆盖范围、因果关系、批处理（Batching） 或实现施加不同的约束。“平衡”不足以宣称 路由器（Router） 具有普遍优越性。

研究锚点：[5]、[6] 理解 混合专家（Mixture of Experts）

21 - OTHER ROUTING FORMULATIONS

Token choice is common, not mandatory

Top-k token-choice routing asks each token which experts it wants. Other work changes the assignment problem itself.

BASE Layers

BASE formulates token-to-expert allocation as a balanced linear assignment problem. Its design enforces equal numbers of tokens per expert and therefore avoids an auxiliary balancing loss in that formulation.

Expert Choice

Expert Choice reverses the direction: each expert selects its top-scoring tokens up to a fixed bucket size. This fixes per-expert capacity by construction, while a token can be selected by a variable number of experts.

Method	Who chooses?	Load property
Token-choice Top-k	token chooses k experts	expert loads can vary
BASE	global balanced assignment	equalized assignment by construction
Expert Choice	expert chooses top tokens	fixed expert bucket, variable experts/-token

These approaches illustrate a broader point: sparse MoE is a family of conditional-computation architectures, not one immutable router algorithm.

Important. Routing formulation changes semantics, optimization, and execution. An alternative that balances loads perfectly may impose different constraints on token coverage, causality, batching, or implementation. “Balanced” is not sufficient to declare one router universally superior.

Research anchors: [5], [6]

22 - 无滴执行

对应原文第 24 页：点击查看相邻原文对照页

22 - DROPLESS EXECUTION 块稀疏 内核（Kernels） 改变容量权衡 动态 MoE 自然会为不同的 专家（Experts） 提供不同的 词元（Token） 计数。致密加速器内核（Kernels）更喜欢常规的、足够大的矩阵（Matrices）。历史上，这种不匹配将运行时推向固定容量、填充或 词元丢弃（Token Dropping）。

MegaBlocks 在 内核（Kernel） 级别攻击不匹配。它表示具有块稀疏 矩阵（Matrix） 操作的 专家（Expert） FFN 计算，可以有效处理可变 专家（Expert） 负载。 MLSys 论文的中央系统声称，这消除了在丢弃溢出 词元（Tokens） 和将 专家（Experts） 填充到通用容量之间的强制选择。

生成的 理解框架（Mental Model） 与 Switch 的静态缓冲区不同：

由 专家（Expert） 块稀疏 专家（Expert） GEMM 路由的 词元（Tokens） 组不相等 词元（Token） 计数保持不相等 不需要对 MoE 进行数学更改。变化在于稀疏、不平衡的 专家（Expert） 批次如何映射到 GPU 计算。

当前的生产框架暴露了这两种行为。 Megatron 核心（Core） 支持容量系数和 词元（Token） 丢弃策略，但其默认 容量因子（Capacity Factor） 当前为 None，记录为 no 词元丢弃（Token Dropping）。

理解框架（Mental Model）.MoE算法设计和MoE 内核（Kernel）设计是耦合的。如果生成的稀疏工作负载与硬件的映射效果不佳，路由器（Router） 在数学上可能很有吸引力，但价格也很昂贵。

研究锚点：[8]、[16] 理解 混合专家（Mixture of Experts）

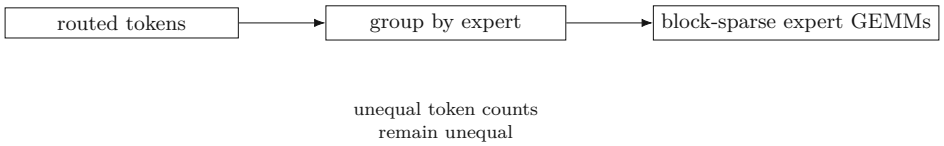
22 - DROPLESS EXECUTION

Block-sparse kernels change the capacity trade-off

A dynamic MoE naturally gives different experts different token counts. Dense accelerator kernels prefer regular, sufficiently large matrices. That mismatch historically pushed runtimes toward fixed capacities, padding, or token dropping.

MegaBlocks attacks the mismatch at the kernel level. It represents expert FFN computation with block-sparse matrix operations that can handle variable expert loads efficiently. The MLSys paper's central systems claim is that this removes the forced choice between dropping overflow tokens and padding experts to a common capacity.

The resulting mental model is different from Switch's static buffers:



No mathematical change to MoE is required. The change is how sparse, imbalanced expert batches are mapped to GPU computation.

Current production frameworks expose both families of behavior. Megatron Core supports capacity factors and token-drop policies, but its default capacity factor is currently `None`, documented as no token dropping.

Mental model. MoE algorithm design and MoE kernel design are coupled. A router can be mathematically attractive yet expensive if the resulting sparse workload maps poorly to hardware.

Research anchors: [8], [16]

23 - 细粒度和 共享专家（Shared Experts）

对应原文第 25 页；点击查看相邻原文对照页

23 - 细粒度和共享专家 TS DeepSeekMoE 更改了 专家（Expert） 银行本身 DeepSeekMoE 针对 专家专门化（Expert Specialization） 提出了两项架构更改。

细粒度专家（Expert）分割从N 专家（Experts） with K active开始，论文概念性地将专家（Experts）更细地分割成mN个更小的专家（Experts）并激活mK。由于每个较小的 专家（Expert） 包含较少的参数，因此设计可以在控制 专家（Expert） 参数和计算预算的同时公开更多组合。

一个纯粹的组合图解说明了为什么 路由（Routing） 空间会扩展。选择 8 个 专家（Experts） 中的 2 个给出

无序 专家（Expert） 集。拆分为 16 个较小的 专家（Experts） 并选择 4 个给出

这并不意味着“智力提高 65 倍”。它仅说明了更大的一组可能的 专家（Expert） 组合。

共享专家（Shared Expert） 隔离 DeepSeekMoE 还保留了一些 专家（Experts） 作为广泛应用的 共享专家（Shared Experts），而路由 专家（Experts） 仍然稀疏。其动机是让共享路径捕获共同知识并减少路由 专家（Experts） 之间的冗余压力。

DeepSeek-V3 使用该系列设计：共享专家（Shared Expert） 计算与选定的路由 专家（Expert） 输出一起添加。

重要： 共享专家（Shared Experts） 更改主动计算会计。 词元（Token） 可以执行其路由专家和 共享专家（Shared Expert） 路径，因此“Top-8”不一定是完整的 FFN 计算故事。

研究锚点：[10]、[11]理解混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

23 - FINE-GRAINED AND SHARED EXPERTS

DeepSeekMoE changes the expert bank itself

DeepSeekMoE proposes two architectural changes aimed at expert specialization.

Fine-grained expert segmentation

Starting from N experts with K active, the paper conceptually splits experts more finely into mN smaller experts and activates mK. Because each smaller expert contains fewer parameters, the design can expose more combinations while controlling expert parameter and computation budgets.

A purely combinatorial illustration shows why the routing space expands. Choosing 2 of 8 experts gives

$$\binom{8}{2} = 28$$

unordered expert sets. Splitting to 16 smaller experts and choosing 4 gives

$$\binom{16}{4} = 1820.$$

This does *not* mean "65x more intelligence." It only illustrates the much larger set of possible expert combinations.

Shared expert isolation

DeepSeekMoE also reserves some experts as shared experts that are applied broadly, while routed experts remain sparse. The motivation is to let shared paths capture common knowledge and reduce redundancy pressure among routed experts.

DeepSeek-V3 uses this family of design: shared expert computation is added alongside selected routed-expert outputs.

Important. Shared experts change active-compute accounting. A token can execute its routed experts *and* shared expert paths, so "Top-8" is not necessarily the complete FFN computation story.

Research anchors: [10], [11]

24 - 辅助无损耗平衡

对应原文第 26 页；点击查看相邻原文对照页

24 - 辅助 Y 无损平衡 DeepSeek-V3 的“无损”实际含义是什么 DeepSeek-V3 使用 sigmoid 词元（Token） 到专家（Expert） 亲和力来表示其路由的 专家（Experts）。在其主要负载均衡机制中，每个专家（Expert）都有一个路由（Routing）biasbi。 Top-K 选择用途

但专家（Expert）的输出权重来自原始亲和力si,t，而不是有偏差的分数。

在 训练（Training） 步骤结束时，对于过载的 专家（Experts），偏置会减小，而对于欠载的 专家（Experts），偏置会增大。

因此，偏差会改变谁被选中，而不直接改变与 专家（Expert） 输出相乘的亲和力 值（Value）。

这就是本文将其主要批处理级方法称为辅助无损失的含义：负载压力是通过动态 路由（Routing） 偏差施加的，而不是通过与语言 模型（Model） 目标竞争的传统批处理级 辅助损失（Auxiliary Loss） 项来施加的。

有一个重要的资格。 DeepSeek-V3 还表示，它使用系数极小的互补序列平衡损失来防止单个序列内部的极端不平衡。

该报告还指出，DeepSeek-V3 在 训练（Training） 期间不会丢弃 词元（Tokens），并描述了节点限制的路由（Routing） 以限制通信。

重要：“DeepSeek-V3 不使用平衡损失”过于宽泛。准确的说法是，其主要的批次级平衡策略是基于偏差且无辅助损失的，同时仍然存在少量互补的序列平衡损失。

研究锚：[11]

了解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

24 - AUXILIARY-LOSS-FREE BALANCING

What DeepSeek-V3 actually means by “loss-free”

DeepSeek-V3 uses sigmoid token-to-expert affinities $s_{i,t}$ for its routed experts. In its main load-balancing mechanism, each expert has a routing bias b_i. Top-K selection uses

$$s_{i,t} + b_i,$$

but the expert’s output weight is derived from the original affinity $s_{i,t}$, not the biased score.

At the end of a training step, the bias is decreased for overloaded experts and increased for underloaded experts. The bias therefore changes *who gets selected* without directly changing the affinity value multiplied into the expert output.

This is the sense in which the paper calls its main batch-level method *auxiliary-loss-free*: load pressure is applied through dynamic routing biases rather than a conventional batch-level auxiliary loss term that competes with the language-model objective.

There is an important qualification. DeepSeek-V3 also states that it uses a complementary *sequence-wise* balance loss with an extremely small coefficient to prevent extreme imbalance inside an individual sequence.

The report additionally states that DeepSeek-V3 does not drop tokens during training, and describes node-limited routing to constrain communication.

Important. “DeepSeek-V3 uses no balancing loss” is too broad. The accurate statement is that its primary batch-level balancing strategy is bias-based and auxiliary-loss-free, while a small complementary sequence-wise balance loss remains.

Research anchors: [11]

25 - 总计 VS 激活参数（Active Parameters）

对应原文第 27 页；点击查看相邻原文对照页

25 - 总参数与活动参数 最常被滥用的 MoE 比较 MoE 模型（Models） 通常用两个参数计数来描述。

总参数（Total Parameters） 所有 模型（Model） 权重存储在密集层、注意力机制（Attention）、嵌入向量（Embeddings）、路由器、共享专家（Shared Experts） 和完整的 RouteDexpert 库中。这些权重对于检查点大小、内存容量、分片、加载和通信都很重要。

激活参数（Active Parameters） per 词元（Token） 根据架构的计算，其计算用于一个 词元（Token） 的前向传播（Forward Pass） 的参数子集。在 MoE 层中，这包括选定的 专家（Expert） 路径，并且可能包括 共享专家（Shared Experts）。 模型（Model） 的密集部分也保持活跃。

在 Mixtral 8x7B 中最容易看出差异。其论文报告称，模型（Model） 可以访问 47B 总参数（Total Parameters），每个 词元（Token） 可以访问 13B 激活参数（Active Parameters）。该论文明确警告说，主动参数分析无法捕获权重内存成本或硬件利用率。

批次级别上出现了进一步的微妙之处。 词元（Token） A 可以激活 专家（Experts） 1 和 4，而 词元（Token） B 可以激活 2 和 7。

在大批量中，可以使用许多或全部 专家（Experts），即使每个单独的 词元（Token） 仅激活少数。

重要： 激活参数（Active Parameters） 不是 模型（Model） 的 内存占用（Memory Footprint），它们也不是 延迟（Latency） 的通用代理。 内核（Kernel） 效率、重量运动、路由（Routing）、通信、批次形状、精度（Precision） 和硬件都很重要。

理解框架（Mental Model）。“每个 词元（Token） 稀疏”并不意味着“推理服务（Serving） 系统中可以缺少大多数 模型（Model） 权重。”研究锚点：[9]、[11]、[14]、[16] 理解 混合专家（Mixture of Experts）

25 - TOTAL VS ACTIVE PARAMETERS

The most abused MoE comparison

MoE models are often described with two parameter counts.

Total parameters

All model weights stored across dense layers, attention, embeddings, routers, shared experts, and the complete routed-expert bank. These weights matter for checkpoint size, memory capacity, sharding, loading, and communication.

Active parameters per token

The subset of parameters whose computations are used for one token's forward pass, according to the architecture's accounting. In an MoE layer this includes selected expert paths and may include shared experts. Dense parts of the model remain active too.

The difference is easiest to see in Mixtral 8x7B. Its paper reports 47B total parameters accessible to the model and 13B active parameters per token. The paper explicitly warns that active-parameter analysis does not capture weight-memory cost or hardware utilization.

A further subtlety appears at batch level. Token A may activate experts 1 and 4 while token B activates 2 and 7. Across a large batch, many or all experts can be used even though each individual token activates only a few.

Important. Active parameters are not the model's memory footprint, and they are not a universal proxy for latency. Kernel efficiency, weight movement, routing, communication, batch shape, precision, and hardware all matter.

Mental model. "Sparse per token" does not mean "most model weights can be absent from the serving system."

Research anchors: [9], [11], [14], [16]

26 - 工作参数示例

对应原文第 28 页；点击查看相邻原文对照页

26 - 工作参数示例 在玩具层中存储更多 专家容量（Expert Capacity），无需更多 专家（Expert） 工作 使用特意简单的两个 矩阵（Matrix） FFN，以便算术透明。忽略偏见。

dmodel = 1024、d ff = 4096 的密集 FFN 有

重量参数。

现在用 8 个较小的 专家（Experts） 创建一个 MoE。每个 专家（Expert） 有 dff = 2048，所以一个 专家（Expert） 有

重量。完整的 专家（Expert） 银行有

专家（Expert） 权重。

如果每个 词元（Token） 使用 Top-2 路由（Routing），则所选的 专家（Expert） 路径接触

专家（Expert） 重量，等于密集玩具 FFN 的重量计数的两个 矩阵（Matrix） 乘积。 路由器（Router） 矩阵（Matrix） 仅添加

该玩具设置中的重量。

因此，这个构造的示例具有 4 倍存储的 专家（Expert） 组参数，同时匹配密集 基线（Baseline） 的理想化 专家（Expert）-矩阵（Matrix） 每个 词元（Token） 参数计数。

重要：这是算术说明，不是 性能（Performance） 保证。真正的法学硕士 FFN 可以使用门控三投影设计、共享专家（Shared Experts）、不同宽度、填充、量化（Quantization）、通信和 融合内核（Fused Kernels）。

相同的参数触摸计数并不意味着相同的运行时间。

研究锚点：[2]、[4]、[9] 理解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

26 - WORKED PARAMETER EXAMPLE

More stored expert capacity without more expert work in a toy layer

Use a deliberately simple two-matrix FFN so the arithmetic is transparent. Ignore biases.

A dense FFN with

$$d_{\text{model}} = 1024, \qquad d_{\text{ff}} = 4096$$

has

$$2 \times 1024 \times 4096 = 8{,}388{,}608$$

weight parameters.

Now create an MoE with 8 smaller experts. Each expert has $d_{\text{ff}} = 2048$, so one expert has

$$2 \times 1024 \times 2048 = 4{,}194{,}304$$

weights. The full expert bank has

$$8 \times 4{,}194{,}304 = 33{,}554{,}432$$

expert weights.

If each token uses Top-2 routing, the selected expert paths touch

$$2 \times 4{,}194{,}304 = 8{,}388{,}608$$

expert weights, equal to the dense toy FFN's weight count for its two matrix multiplies. The router matrix adds only

$$1024 \times 8 = 8192$$

weights in this toy setup.

So this constructed example has 4x the stored expert-bank parameters while matching the dense baseline's idealized expert-matrix parameter count per token.

Important. This is an arithmetic illustration, not a performance guarantee. Real LLM FFNs may use gated three-projection designs, shared experts, different widths, padding, quantization, communication, and fused kernels. Equal parameter-touch counts do not imply equal runtime.

Research anchors: [2], [4], [9]

27 - 训练（Training） VS 推理（Inference）

对应原文第 29 页；点击查看相邻原文对照页

27 - 训练（Training） VS 推理（Inference） 路由器（Router） 存在于两个阶段，但问题不同。在 训练（Training） 期间，许多 词元（Tokens） 一起路由，梯度流经选定的 专家（Expert） 计算和 路由器（Router） 权重。 专家（Expert） 仅从在该步骤中使用它的 词元（Tokens） 接收主任务 梯度（Gradient）。平衡和稳定性目标（如果存在）添加 路由器（Router） 特定的 训练（Training） 信号。

这会创建批次级动态：专家（Expert） 利用率会影响每个 专家（Expert） 学习的量，而当前 路由器（Router） 会影响每个 专家（Expert） 看到的数据分布。

在 推理（Inference） 期间，任务有所不同。没有 反向传播（Backward Pass） 和 训练（Training），仅辅助损耗未优化，但 模型（Model） 仍必须计算 路由（Routing） 决策、分派（Dispatch） 词元（Token） 状态、执行选定的 专家（Experts） 和 合并（Combine） 输出。

对于自回归解码，每个序列通常为每个解码步骤贡献一个新的 词元（Token）。如果没有 批处理（Batching） 许多请求，单个 专家（Expert） 一次只能接收很少的 词元（Tokens），从而使得 专家（Expert） GEMM 较小且效率低下。大批量 推理服务（Serving） 可以改进 算术强度（Arithmetic Intensity），但也会使 路由（Routing）/负载模式更加重要。

路由（Routing） 也是特定于层的。生成的 词元（Token） 可以在早期层选择一组 专家（Experts），并在稍后选择另一组，因为它的 隐藏表示（Hidden Representation） 发生了变化。

重要：“MoE 训练（Training） 很稀疏，因此 推理（Inference） 自动很快”不是有效的推论。仅当 推理服务（Serving） 实现可以有效地利用生成的 路由（Routing） 模式时，稀疏算术才有用。

研究锚点：[2]、[9]、[11]、[16] 理解 混合专家（Mixture of Experts）

27 - TRAINING VS INFERENCE

The router exists in both phases, but the problems differ

During training, many tokens are routed together and gradients flow through the selected expert computations and router weights. An expert receives main-task gradient only from tokens that use it on that step. Balancing and stability objectives, when present, add router-specific training signals.

This creates batch-level dynamics: expert utilization affects how much each expert learns, and the current router affects the data distribution seen by each expert.

During inference, the task is different. There is no backward pass and training-only auxiliary losses are not optimized, but the model still has to compute routing decisions, dispatch token states, execute selected experts, and combine outputs.

For autoregressive decode, each sequence typically contributes one new token per decode step. Without batching many requests, an individual expert can receive very few tokens at a time, making expert GEMMs small and inefficient. Large serving batches can improve arithmetic intensity but also make routing/load patterns more important.

Routing is also layer-specific. A generated token can select one set of experts at an early layer and another set later because its hidden representation changes.

Important. "MoE training is sparse, so inference is automatically fast" is not a valid deduction. Sparse arithmetic helps only if the serving implementation can exploit the resulting routing pattern efficiently.

Research anchors: [2], [9], [11], [16]

28 - 专家并行（Expert Parallelism）

对应原文第 30 页；点击查看相邻原文对照页

28 - EXPER T 并行 分派（Dispatch）、计算、合并（Combine） 大型 专家（Expert） 库可能不适合在一个 GPU 上运行，或者可能效率不高。 专家并行（Expert Parallelism） 跨设备对 专家（Experts） 进行分区。 因此，路由器（Router） 可以选择物理上远离 词元（Token） 当前设备的 专家（Expert）。

常见的执行模式是：

1. 计算路由（Routing）指数和权重；

2. 按目的地 专家（Expert） 排列/分组 词元（Token） 状态；

3. 将 词元（Tokens） 与托管这些 专家（Experts） 的设备进行通信；

4. 执行 专家（Expert） FFN，通常作为分组或融合的 GEMM；

5. 通讯专家（Expert）输出回来；

6. 取消排列并按原始 词元（Token） 顺序输出 合并（Combine）。

路由排列 全对全通信（All-to-all） 专家（Experts） 全对全通信（All-to-all） 合并（Combine） 当前威震天 核心（Core） 文档 全对全通信（All-to-all）、AllGather 和灵活的 词元（Token）-分派（Dispatch） 后端。其 全对全通信（All-to-all） 调度程序显式执行 分派（Dispatch） 到 专家（Expert） 等级、专家（Expert） 主重新排序，然后执行 合并（Combine） 通信和取消排列。

重要： 专家并行（Expert Parallelism） 不是 MoE 的定义。它是 专家（Expert） 银行的分布式执行策略。 小型 MoE 可以在一台设备上运行，无需 GPU 间 专家（Expert） 通信。

研究锚点：[3]、[11]、[16] 理解 混合专家（Mixture of Experts）

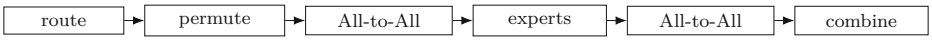
28 - EXPERT PARALLELISM

Dispatch, compute, combine

A large expert bank may not fit, or may not be efficient, on one GPU. Expert parallelism partitions experts across devices. The router can therefore select an expert that is physically remote from the token's current device.

A common execution pattern is:

1. compute routing indices and weights;
2. permute/group token states by destination expert;
3. communicate tokens to devices that host those experts;
4. execute expert FFNs, often as grouped or fused GEMMs;
5. communicate expert outputs back;
6. unpermute and combine outputs in original token order.



Current Megatron Core documents All-to-All, AllGather, and flexible token-dispatch backends. Its All-to-All dispatcher explicitly performs dispatch to expert ranks, expert-major reordering, then a combine communication and unpermutation.

Important. Expert parallelism is not the definition of MoE. It is a distributed execution strategy for the expert bank. A small MoE can run on one device with no inter-GPU expert communication.

Research anchors: [3], [11], [16]

29 - 性能（Performance）

对应原文第 31 页；点击查看相邻原文对照页

29 - 性能（Performance） 为什么更少的 激活参数（Active Parameters） 仍然会运行得很差 稀疏 专家（Expert） 计算会更改 性能（Performance） 模型（Model） 而不是删除它。多种成本相互竞争。

专家（Expert） 通用矩阵乘法（GEMM） 大小 对于许多 专家（Experts），词元（Tokens） 被分成更小的每个 专家（Expert） 批次。微小或不规则的 矩阵（Matrices） 可能无法充分利用加速器。 2017 年 MoE 的论文将其称为“收缩批次问题”。

通信 专家并行（Expert Parallelism） 将 词元（Token） 状态移至 专家（Expert） 器件并输出回来。如果计算/通信比率较差，全对全通信（All-to-all） 通信可以与 专家（Expert） 计算相当。

重量移动 即使每个 词元（Token） 仅使用几个 专家（Experts），一个批次也可能会涉及多个 专家（Experts）。因此，重量 驻留状态（Residency） 和 内存带宽（Memory Bandwidth） 仍然很重要，特别是在小批量情况下。

负载不平衡 如果工作没有有效重叠，最慢的 专家（Expert）/rank 可以确定暴露的步骤时间。

路由（Routing） 和排列开销 分数计算、Top-k、元数据构建、排序/排列和 合并（Combine） 操作都会消耗时间。

现代系统通过分组 GEMM、块稀疏 内核（Kernels）、拓扑感知 路由（Routing）、融合 分派（Dispatch） 以及计算/通信重叠进行响应。例如，DeepSeek-V3 明确地将 路由（Routing） 与跨节点通信共同设计，并限制 词元（Token） 可以联系的节点数量。

理解框架（Mental Model）.活动参数计数通过权重描述数学路径长度。 性能（Performance） 取决于硬件是否可以通过大型、本地、平衡良好的操作执行该路径。

研究锚点：[2]、[8]、[9]、[11]、[16] 理解 混合专家（Mixture of Experts）

29 - PERFORMANCE

Why fewer active parameters can still run badly

Sparse expert computation changes the performance model rather than deleting it. Several costs compete.

Expert GEMM size

With many experts, tokens are split into smaller per-expert batches. Tiny or irregular matrices can underutilize accelerators. The 2017 MoE paper called this the shrinking-batch problem.

Communication

Expert parallelism moves token states to expert devices and outputs back. All-to-All communication can become comparable to expert computation if the compute/communication ratio is poor.

Weight movement

Even when only a few experts are used per token, a batch can touch many experts. Weight residency and memory bandwidth therefore remain important, especially at low batch sizes.

Load imbalance

The slowest expert/rank can determine the exposed step time if work is not effectively overlapped.

Routing and permutation overhead

Score computation, Top-k, metadata construction, sorting/permutation, and combine operations all consume time.

Modern systems respond with grouped GEMMs, block-sparse kernels, topology-aware routing, fused dispatch, and computation/communication overlap. DeepSeek-V3, for example, explicitly co-designs routing with cross-node communication and limits the number of nodes a token can contact.

Mental model. Active parameter count describes the mathematical path length through weights. Performance depends on whether hardware can execute that path with large, local, well-balanced operations.

Research anchors: [2], [8], [9], [11], [16]

30 - 专家（Expert） 专业化

对应原文第 32 页；点击查看相邻原文对照页

30 - 专家专业化 专家（Experts） 是否会变成“数学 专家（Experts）”和“代码 专家（Experts）”？

将教育部变成一个人类故事是很诱人的：一个 专家（Expert） 学习数学，另一个学习代码，另一个学习法语。研究并不能证明其作为一般的 理解框架（Mental Model） 是合理的。

最初的稀疏门控 MoE 报告了其循环语言 模型（Models） 的专业化模式。但专业化在不同架构和层之间可能看起来有所不同。

Mixtral 对 The Pile 的跨域进行了 路由（Routing） 分析。作者报告说，大多数数据集没有明显的域级分配模式：ArXiv、PubMed、哲学和其他数据集具有类似的 专家（Expert） 分配分布。他们确实观察到了更结构化的语法行为，例如缩进或某些 词元（Token） 模式的重复分配，以及某些层中的大量局部性。

ST-MoE 还发现专业化取决于背景。其分析报告 编码器（Encoder） 专家（Experts） 中的专业化模式，而 解码器（Decoder） 专家（Experts） 没有表现出相同的清晰行为，并且多语言专业化并没有简化为简单的“每个 专家（Expert） 一种语言”分区。

这使得三个概念值得区分：

- 路由（Routing） 频率：选择哪个专家（Experts）；

- 功能专业化：专家（Experts） 是否执行明显不同的转换；

- 人类可解释的专业化：这些差异是否清晰地映射到“数学”等标签。

重要：彩色 专家（Expert）-路由（Routing） 图是关于选择模式的 证据（Evidence），而不是自动 证据（Evidence）（专家（Expert） 存储干净的语义域）。

研究锚点：[2]、[7]、[9] 理解 混合专家（Mixture of Experts）

30 - EXPERT SPECIALIZATION

Do experts become “math experts” and “code experts”?

It is tempting to turn MoE into a human story: one expert learns mathematics, another learns code, another learns French. Research does not justify that as a general mental model.

The original sparsely-gated MoE reported specialization patterns in its recurrent language models. But specialization can look different across architectures and layers.

Mixtral performed a routing analysis across domains from The Pile. The authors reported no obvious domain-level allocation pattern for most datasets: ArXiv, PubMed, philosophy, and others had similar expert-assignment distributions. They did observe more structured syntactic behavior, such as recurring assignments for indentation or certain token patterns, and substantial locality in some layers.

ST-MoE also found that specialization depended on context. Its analysis reports specialization patterns in encoder experts while decoder experts did not exhibit the same clear behavior, and multilingual specialization did not reduce to simple “one language per expert” partitions.

This makes three concepts worth separating:

- **routing frequency**: which experts get selected;
- **functional specialization**: whether experts perform measurably different transformations;
- **human-interpretable specialization**: whether those differences map cleanly to labels such as “math.”

Important. A colorful expert-routing plot is evidence about selection patterns, not automatically evidence that an expert stores a clean semantic domain.

Research anchors: [2], [7], [9]

31 - 现代 模型（Models）

对应原文第 33 页；点击查看相邻原文对照页

31 - MODERN 模型（Models） 三种设计，三种不同的 MoE 选择 现代开放 模型（Models） 明确表示“MoE”并不意味着一个 专家（Expert） 计数或 路由（Routing） 规则。

模型（Model） 总计/活动 专家（Expert） 路由（Routing） 选定的架构事实 Mixtral 8x7B 47B / 13B 8 专家（Experts）、Top-2 FFN 专家（Experts）、Softmax 归一化（Softmax） 超过 Top-2 logits DeepSeek-V3 671B / 37B 256路由，Top-8细粒度路由专家（Experts） + 共享专家（Shared Expert），sigmoid亲和力 Qwen3-30B-A3B 30.5B / 3.3B 128 专家（Experts），Top-8官方模型（Model）卡报告48层和8个激活的专家（Experts）这些标题参数计数来自每个 模型（Model） 的作者，应在该架构的会计中进行解释。它们是有用的描述，不能直接比较 延迟（Latency） 测量值。

Mixtral 的论文明确指出，主动参数分析不包括内存成本和硬件利用率。 DeepSeek-V3 将其 专家（Expert） 架构与节点限制的 路由（Routing） 和自定义通信相结合。

Qwen3 的 模型（Model） 卡报告的 专家（Expert） 库比 Mixtral 大得多，每个 词元（Token） 有 8 个活动 专家（Experts）。

重要：不存在“更多 专家（Experts）”或“更少 激活参数（Active Parameters）”产生更快或更好的 模型（Model） 的单调规则。 专家（Expert） 大小、密集主干、路由（Routing）、精度（Precision）、并行性和实现都在比较中发生了变化。

研究锚点：[9]、[11]、[14] 理解 混合专家（Mixture of Experts）

31 - MODERN MODELS

Three designs, three different MoE choices

Modern open models make it clear that “MoE” does not imply one expert count or routing rule.

Model	Total / active	Expert routing	Selected architecture facts
Mixtral 8x7B	47B / 13B	8 experts, Top-2	FFN experts, softmax over Top-2 logits
DeepSeek-V3	671B / 37B	256 routed, Top-8	fine-grained routed experts + shared expert, sigmoid affinity
Qwen3-30B-A3B	30.5B / 3.3B	128 experts, Top-8	official model card reports 48 layers and 8 activated experts

These headline parameter counts come from each model’s authors and should be interpreted within that architecture’s accounting. They are useful descriptions, not directly comparable latency measurements.

Mixtral’s paper explicitly notes that active-parameter analysis does not include memory cost and hardware utilization. DeepSeek-V3 couples its expert architecture to node-limited routing and custom communication. Qwen3’s model card reports a much larger expert bank than Mixtral with eight active experts per token.

Important. There is no monotonic rule that “more experts” or “fewer active parameters” produces a faster or better model. Expert size, dense backbone, routing, precision, parallelism, and implementation all change the comparison.

Research anchors: [9], [11], [14]

32 - 透明实施

对应原文第 34 页；点击查看相邻原文对照页

32 - 透明实施 教学 Top-2 词元（Token） 选择 路由器（Router） 以下 PyTorch 风格的草图展示了概念阶段。它是原始的教学代码，而不是教育部的高效生产实施。

代码注释： x: [词元（Tokens）, d_模型（Model）]

logits = x @ 路由器（Router） _weight # [词元（Tokens）, 专家（Experts）] probs = logits.Softmax 归一化（Softmax）(dim=-1) top_p, top_i = probs.topk(k=2, dim=-1) top_p = top_p / top_p.总和（暗淡=-1，keepdim=True）

掩码（Mask） = (top_i[:, 槽] == 专家（Expert） _id)

y = 专家（Expert）(x[掩码（Mask）]) out[掩码（Mask）] += top_p[掩码（Mask）, slot, None] * y 它使数学步骤可见：

分数→ Top-2 指数→ 专家（Expert） 看涨期权→ 加权 合并（Combine）。

代码故意隐藏的是大部分硬工程：容量策略、稳定的 路由（Routing） 精度（Precision）、分布式 分派（Dispatch）、词元（Token） 排列、分组 GEMM、通信重叠、共享专家（Shared Experts）、负载平衡损耗、训练（Training） 噪声和专用 内核（Kernels）。

大规模的嵌套 Python 循环将是灾难性的。生产系统将许多 词元（Tokens） 分组为相同的 专家（Expert），并调用批处理/融合的 专家（Expert） 计算。

重要：此示例还重新规范化了选定的 Softmax 归一化（Softmax） 概率。这是一种有效的教学惯例，而不是每个 模型（Model） 的通用 路由器（Router） 定义。

研究锚点：[2]、[9]、[16] 理解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

32 - TRANSPARENT IMPLEMENTATION

A pedagogical Top-2 token-choice router

The following PyTorch-style sketch exposes the conceptual stages. It is original pedagogical code, not an efficient production MoE implementation.

```
# x: [tokens, d_model]
logits = x @ router_weight          # [tokens, experts]
probs = logits.softmax(dim=-1)

top_p, top_i = probs.topk(k=2, dim=-1)
top_p = top_p / top_p.sum(dim=-1, keepdim=True)

out = torch.zeros_like(x)
for slot in range(2):
    for expert_id, expert in enumerate(experts):
        mask = (top_i[:, slot] == expert_id)
        if mask.any():
            y = expert(x[mask])
            out[mask] += top_p[mask, slot, None] * y
```

It makes the mathematical steps visible:

$$\text{scores} \to \text{Top-2 indices} \to \text{expert calls} \to \text{weighted combine.}$$

What the code deliberately hides is most of the hard engineering: capacity policies, stable routing precision, distributed dispatch, token permutation, grouped GEMMs, communication overlap, shared experts, load-balancing losses, training noise, and specialized kernels.

The nested Python loops would be disastrous at scale. Production systems group many tokens for the same expert and invoke batched/fused expert computation instead.

Important. This example also renormalizes the selected softmax probabilities. That is one valid pedagogical convention, not the universal router definition for every model.

Research anchors: [2], [9], [16]

33 - 实施错误

对应原文第 35 页；点击查看相邻原文对照页

33 - 实施过程中出现了一些看似合理的错误

1.将Top-k指数视为可微概率。

成员资格的决定是离散的。梯度根据 路由器（Router） 设计流经选定的权重和辅助目标，而不是通过假想的平滑 argmax。

2. 假设所有路由器都使用Softmax 归一化（Softmax）。

DeepSeek-V3 对路由 专家（Experts） 使用 sigmoid 关联性。 路由器（Router） 归一化（Normalization） 和权重必须与 模型（Model） 匹配。

3. 对错误的集合进行标准化。

Softmax 归一化（Softmax） 在所有 专家（Experts） 上进行掩码，Softmax 归一化（Softmax） 在 Top-k 掩码之后，以及仅在选定的 sigmoid 亲和力中的 归一化（Normalization） 是不同的计算。

4. 将 词元（Token） 计数与赋值计数混淆。

Top-k 从 T 词元（Tokens） 创建 tokT 词元（Token）-专家（Expert） 分配。容量公式必须符合 路由器（Router）/运行时约定。

5. 分派（Dispatch）之后丢失原始词元（Token）订单。

专家（Expert）-major 执行需要足够的元数据，以便 合并（Combine） 输出回正确的 词元（Token） 行和 路由（Routing） 槽。

6. 在错误的范围内使用 专家（Expert） 的负载统计数据。

每个设备、逐组（Per-group）、每个序列和全局批次平衡可能有所不同。

7. 假设 激活参数（Active Parameters） 等于 驻留（Resident） 权重。

即使 词元（Token） 仅使用一个子集，推理服务（Serving） 也可能需要整个分片 专家（Expert） 库。

8. 在不命名硬件和批次形状的情况下比较 MoE 速度。

稀疏执行效率对 专家（Expert） 批大小（Batch Size）、互连、精度（Precision） 和 内核（Kernels） 高度敏感。

研究锚点：[4]、[7]、[11]、[16] 理解 混合专家（Mixture of Experts）

33 - IMPLEMENTATION MISTAKES

Bugs that still look plausible

1. Treating Top-k indices as differentiable probabilities.

The membership decision is discrete. Gradients flow through selected weights and auxiliary objectives according to the router design, not through an imaginary smooth argmax.

2. Assuming all routers use softmax.

DeepSeek-V3 uses sigmoid affinities for routed experts. Router normalization and weighting must match the model.

3. Normalizing over the wrong set.

Softmax over all experts followed by masking, softmax after Top-k masking, and normalization only among selected sigmoid affinities are different computations.

4. Confusing token count with assignment count.

Top-k creates up to kT token-expert assignments from T tokens. Capacity formulas must match the router/runtime convention.

5. Losing original token order after dispatch.

Expert-major execution requires metadata sufficient to combine outputs back into the correct token rows and routing slots.

6. Using one expert's load statistics at the wrong scope.

Per-device, per-group, per-sequence, and global-batch balance can differ.

7. Assuming active parameters equal resident weights.

Serving may need the whole sharded expert bank even though a token uses only a subset.

8. Comparing MoE speed without naming hardware and batch shape.

Sparse execution efficiency is highly sensitive to expert batch size, interconnect, precision, and kernels.

Research anchors: [4], [7], [11], [16]

34 - 常见的误解

对应原文第 36 页：点击查看相邻原文对照页

34 - 常见误解 创建错误 理解框架（Mental Model） 的快捷方式“专家（Expert） 是一个完整的 LLM。”在现代 Transformer 模型（Transformer） MoE 中通常为 false。 专家（Experts） 通常是选定层内的 FFN/MLP 子网。

“教育部只运行一个很小的 模型（Model）。” 词元（Token） 激活稀疏 专家（Expert） 路径，但密集/共享参数仍然运行，并且完整的 专家（Expert） 库仍然存在。

“总计 100B，活动 10B 意味着 10B 内存。” 激活参数（Active Parameters） 不是检查点或 驻留（Resident） 权重内存。

“MoE 始终使用 Top-2。” Switch是Top-1，Mixtral Top-2，Qwen3和DeepSeek-V3使用更大的路由（Routing）预算。

“教育部总是会掉落 词元（Tokens）。”不。丢弃是一项容量策略。 MegaBlocks 和当前的无滴运行时间却显示出不同的情况。

“负载均衡（Load Balancing） 意味着每个 专家（Expert） 应该是相同的。”平衡涉及利用率/负载。有用的功能差异仍然可能出现。

“每个 专家（Expert） 学习一个领域。” 路由（Routing） 分析不支持将此作为通用解释。

“更多的 专家（Experts） 使 推理（Inference） 更快。”更多 专家（Experts） 会增加容量，但会恶化 路由（Routing）、体重移动和沟通。

“MoEsparsity就是稀疏注意力。” MoE 稀疏参数/专家（Expert） 激活（Activation）。 稀疏注意力（Sparse Attention） 更改了 查询（Query）-键（Key） 连接性。

研究锚点：[4]、[8]、[9]、[11]、[14] 理解 混合专家（Mixture of Experts）

34 - COMMON MISCONCEPTIONS

Shortcuts that create the wrong mental model

“An expert is a whole LLM.”	Usually false in modern Transformer MoEs. Experts are typically FFN/MLP subnetworks inside selected layers.
“MoE runs only a tiny model.”	A token activates sparse expert paths, but dense/shared parameters still run and the full expert bank still exists.
“100B total and 10B active means 10B memory.”	Active parameters are not checkpoint or resident-weight memory.
“MoE always uses Top-2.”	Switch is Top-1, Mixtral Top-2, Qwen3 and DeepSeek-V3 use larger routing budgets.
“MoE always drops tokens.”	No. Dropping is one capacity policy. MegaBlocks and current dropless runtimes show otherwise.
“Load balancing means every expert should be identical.”	Balance concerns utilization/load. Useful functional differences can still emerge.
“Each expert learns one domain.”	Routing analyses do not support this as a universal interpretation.
“More experts makes inference faster.”	More experts increases capacity but can worsen routing, weight movement, and communication.
“MoE sparsity is sparse attention.”	MoE sparsifies parameter/expert activation. Sparse attention changes query-key connectivity.

Research anchors: [4], [8], [9], [11], [14]

35 - 完整的心理 模型（Model）

对应原文第 37 页；点击查看相邻原文对照页

35 - 一条链中的完整 模型（Model） 混合专家（Mixture of Experts） 现代稀疏 Transformer 模型（Transformer） MoE 可以被记住为九个链接的想法。

1. 从 FFN 开始。专家（Expert） 库通常替换密集的 前馈（Feed-forward） 子层，而不是 注意力机制（Attention）。

2. 得分。 路由器（Router） 将每个 词元（Token） 表示（Representation） 映射到 专家（Expert） 分数或亲和力。

3. 选择。稀疏 路由（Routing） 规则选择 专家（Experts） 的一个小子集。

4. 计算。仅针对该 词元（Token） 执行选定的路由 专家（Experts），以及架构定义的任何共享路径。

5.合并（Combine）。 路由器（Router） 派生权重将选定的 专家（Expert） 输出合并回一个 词元（Token） 表示（Representation）。

6.平衡。 训练（Training） 必须防止病态的 专家（Expert） 使用不足/过度使用，而不让平衡压力主导任务目标。

7. 稳定。 路由器（Router） 数值稳定性与负载平衡不同； z 损失等技术针对的是不同的故障模式。

8.分派（Dispatch）。在 专家（Expert） 并行系统中，词元（Token） 状态移动到 专家（Expert） 设备并输出返回，通常通过 全对全通信（All-to-all） 通信。

9.测量正确的资源。总参数（Total Parameters）、激活参数（Active Parameters）、浮点运算次数（FLOPs）、内存容量、内存流量（Memory Traffic）、通信和实际耗时（Wall-clock Time）不可互换。

MoE 的持久承诺是条件计算：增加 模型（Model） 可用的参数容量，而不强制每个 词元（Token） 通过每个 专家（Expert） 参数。其持久的困难在于条件计算会产生动态的、不平衡的、分布式的工作，而密集的硬件无法很好地自动执行。

如果这两种说法结合在一起，大多数教育部的说法就会变得更容易评估。该架构既不是“以微小的价格获得巨大的 模型（Model）”，也不仅仅是“带有 路由器（Router） 的多个 FFN”。它是一个跨越优化、数值稳定性、稀疏计算、内存、通信和 模型（Model） 架构的协同设计问题。

研究锚点：[1]-[16]理解混合专家（Mixture of Experts）

35 - THE COMPLETE MENTAL MODEL

Mixture of Experts in one chain

A modern sparse Transformer MoE can be remembered as nine linked ideas.

1. **Start with the FFN.** The expert bank usually replaces a dense feed-forward sublayer, not attention.
2. **Score.** A router maps each token representation to expert scores or affinities.
3. **Select.** A sparse routing rule chooses a small subset of experts.
4. **Compute.** Only selected routed experts execute for that token, plus any shared paths the architecture defines.
5. **Combine.** Router-derived weights merge selected expert outputs back into one token representation.
6. **Balance.** Training must prevent pathological expert under-use/over-use without letting balancing pressure dominate the task objective.
7. **Stabilize.** Router numerical stability is distinct from load balance; techniques such as z-loss target a different failure mode.
8. **Dispatch.** In expert-parallel systems, token states move to expert devices and outputs return, often through All-to-All communication.
9. **Measure the right resource.** Total parameters, active parameters, FLOPs, memory capacity, memory traffic, communication, and wall-clock time are not interchangeable.

MoE's durable promise is conditional computation: increase the parameter capacity available to the model without forcing every token through every expert parameter. Its durable difficulty is that conditional computation produces dynamic, imbalanced, distributed work that dense hardware does not execute automatically well.

If those two statements stay together, most MoE claims become easier to evaluate. The architecture is neither "a giant model for the price of a tiny one" nor merely "several FFNs with a router." It is a co-design problem spanning optimization, numerical stability, sparse computation, memory, communication, and model architecture.

Research anchors: [1]-[16]

参考

对应原文第 38 页；点击查看相邻原文对照页

参考文献 主要论文和当前实施来源

1. R. A. Jacobs、M. I. Jordan、S. J. Nowlan、G. E. Hinton。局部 专家（Experts） 的自适应混合物。神经计算 3(1):79-87,

1991.DOI：10.1162/neco.1991.3.1.79。

2. N. Shazeer 等人的《超大神经网络：稀疏门控 混合专家（Mixture-of-Experts） 层》。 arXiv：1701.06538，2017。

3. D. Lepikhin 等人 GShard：缩放（Scaling） 具有条件计算和自动分片功能的巨型 模型（Models）。 arXiv:2006.16668, 2020; ICLR 2021。

4. W. Fedus、B. Zoph、N. Shazeer.Switch 变压器：缩放（Scaling） 到万亿参数 模型（Models），具有简单高效的稀疏性。

JMLR 23(120):1-39, 2022。

5. M. Lewis 等人的基础层：简化大型稀疏 模型（Models） 的 训练（Training）。 arXiv：2103.16716，2021。

6. Y. Zhou 等人.混合专家（Mixture-of-Experts） 和 专家（Expert） 选择 路由（Routing）。 NeurIPS 2022。 arXiv：2202.09368。

7. B. Zoph 等人 ST-MoE：设计稳定且可转移的稀疏 专家（Expert） 模型（Models）。 arXiv：2202.08906，2022。

8. T. Gale、D. Narayanan、C. Young、M. Zaharia.MegaBlocks：高效稀疏 训练（Training） 和 混合专家（Mixture-of-Experts）。 MLSys 论文集，卷。 2023 年 5 月。arXiv：2211.15841。

9. A. Q. Jiang et al.Mixtral of 专家（Experts）。 arXiv：2401.04088，2024。

10. D. Dai 等人。DeepSeekMoE：以 混合专家（Mixture-of-Experts） 语言 模型（Models） 走向终极 专家专门化（Expert Specialization）。 arXiv:2401.06066,

11.DeepSeek-AI。 DeepSeek-V3 技术报告。 arXiv：2412.19437v2，2025；参考官方 DeepSeek-V3 存储库/配置

12. N. Muennighoff 等人。OLMoE：开放 混合专家（Mixture-of-Experts） 语言 模型（Models）。 arXiv：2409.02060，2024。

13. Qwen团队.Qwen3技术报告。 arXiv：2505.09388，2025。

14. Qwen Team.Qwen3-30B-A3B官方模型（Model）卡。当前官方 模型（Model） 元数据参考了 2026 年 9 月的总/激活参数和 专家（Expert） 计数。

15. Mistral AI.Mixtral 8x7B 模型（Model） 发布/源材料。仅在与主要 Mixtral 纸一致的情况下使用。

16. NVIDIA Megatron 核心（Core） 文档。MoE 词元（Token） 调度程序、专家并行（Expert Parallelism）、路由器（Router）/容量配置。当前文档查阅于 2026 年 9 月。

了解 混合专家（Mixture of Experts）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

REFERENCES

Primary papers and current implementation sources

1. R. A. Jacobs, M. I. Jordan, S. J. Nowlan, G. E. Hinton. *Adaptive Mixtures of Local Experts.* Neural Computation 3(1):79-87, 1991. DOI: 10.1162/neco.1991.3.1.79.
2. N. Shazeer et al. *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.* arXiv:1701.06538, 2017.
3. D. Lepikhin et al. *GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding.* arXiv:2006.16668, 2020; ICLR 2021.
4. W. Fedus, B. Zoph, N. Shazeer. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.* JMLR 23(120):1-39, 2022.
5. M. Lewis et al. *BASE Layers: Simplifying Training of Large, Sparse Models.* arXiv:2103.16716, 2021.
6. Y. Zhou et al. *Mixture-of-Experts with Expert Choice Routing.* NeurIPS 2022. arXiv:2202.09368.
7. B. Zoph et al. *ST-MoE: Designing Stable and Transferable Sparse Expert Models.* arXiv:2202.08906, 2022.
8. T. Gale, D. Narayanan, C. Young, M. Zaharia. *MegaBlocks: Efficient Sparse Training with Mixture-of-Experts.* Proceedings of MLSys, Vol. 5, 2023. arXiv:2211.15841.
9. A. Q. Jiang et al. *Mixtral of Experts.* arXiv:2401.04088, 2024.
10. D. Dai et al. *DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models.* arXiv:2401.06066, 2024.
11. DeepSeek-AI. *DeepSeek-V3 Technical Report.* arXiv:2412.19437v2, 2025; official DeepSeek-V3 repository/configuration consulted for architecture details.
12. N. Muennighoff et al. *OLMoE: Open Mixture-of-Experts Language Models.* arXiv:2409.02060, 2024.
13. Qwen Team. *Qwen3 Technical Report.* arXiv:2505.09388, 2025.
14. Qwen Team. *Qwen3-30B-A3B official model card.* Current official model metadata consulted September 2026 for total/activated parameters and expert counts.
15. Mistral AI. *Mixtral 8x7B model release/source materials.* Used only where consistent with the primary Mixtral paper.
16. NVIDIA Megatron Core documentation. *MoE token dispatchers, expert parallelism, router/capacity configuration.* Current documentation consulted September 2026.